

Machine Learning

"Field of study that gives computers the ability to learn without being explicitly programmed."

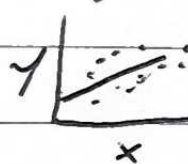
- Arthur Samuel 1959

Supervised learning

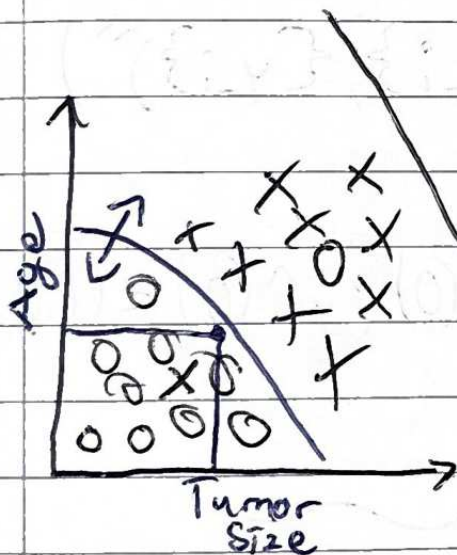
$x \rightarrow y$
Input feature Label target

Regression

ratio of x to y

 "data has right answers"

App	Input (X)	Output (y)
spam filter	email	spam? (0/1)
speech rec	audio	text transcripts
translate	English	Spanish
adverts	ad, user info	click? (0/1)
self-driving	img, radar	pos of other cars



Example:
Cancer

Classification

predict categories
→ small nr of possible outputs

O → benign X → malignant

linear regression

Training set

Learning Algorithm

$x \rightarrow \boxed{f} \rightarrow y$

feature

model

prediction
(estimated y)
↑
target

$n \in \text{hat}$

Unsupervised Learning

Find something interesting
in unlabeled data → patterns

Clustering → Grouping together

Anomaly detection → Find unusual data points

Dimensionality reduction → compress data using fewer hrs

linear regression (with one variable)

$$f_{w,b}(x) = wx + b$$

Univariate!

Example python code

import matplotlib.pyplot as plt
import numpy as np

```
x_train = np.array([1.0, 2.0])  
y_train = np.array([300.0, 500.0])  
print("{x_train.shape}")
```

→ (2, 1)

```
→  
[  
    x_i = x_train[i]  
    y_i = y_train[i]  
    i = 0  
]
```

```
print(f"(x^{i}), y^{i} = ({x_i}, {y_i})")
```

↓
 $x^0, y^0 = (1.0, 300.0)$

#plot the data pts

plt.scatter(x_train, y_train)

Cost functions

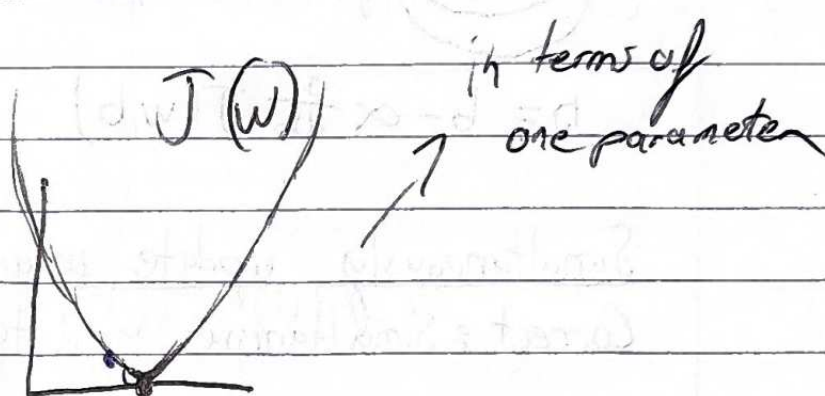
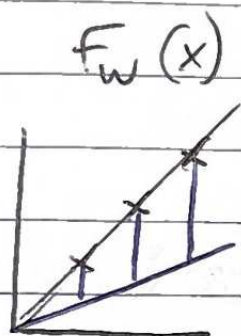
$$J(w,b) = \frac{1}{2m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2 \quad \text{for } m \text{ nr of training examples}$$

↑
heater

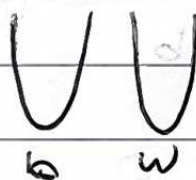
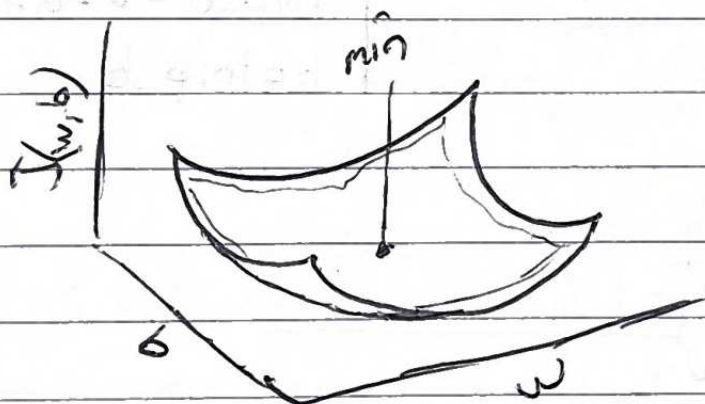
error

Squared error cost function

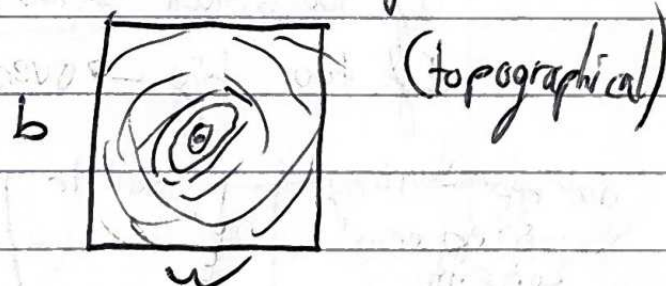
↳ Goal is minimize $J(w,b)$ aka err



$w=1$



contour plot



Gradient descent

used on any function

$$\min_{w_1, \dots, w_n, b} J(w_1, w_2, \dots, w_n, b)$$

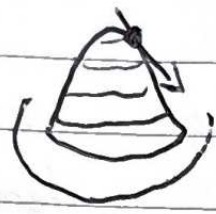
Outline: Start at some w, b (maybe set $w=0, b=0$)

Keep changing w, b to reduce $J(w,b)$

Until we set at a minimum

Gradient descent Algorithm

$$w = w - \underbrace{\alpha}_{\substack{\text{learning rate} \\ \text{(steps size)}}} \underbrace{\frac{d}{dw} J(w,b)}_{\substack{\text{derivative} \\ \text{of cost function (slope)}}$$


 this is essentially what the formula says, size of arrow is LR ^{direction/slope is derivative}

$$b = b - \alpha \frac{d}{db} J(w,b)$$

Simultaneously update w and b

Correct: Simultaneous update

Incorrect:

repeat until
convergence

$$\left. \begin{aligned} \text{tmp_w} &= w - \alpha \frac{\partial}{\partial w} J(w,b) \\ \text{tmp_b} &= b - \alpha \frac{\partial}{\partial b} J(w,b) \\ w &= \text{tmp_w} \\ b &= \text{tmp_b} \end{aligned} \right\}$$

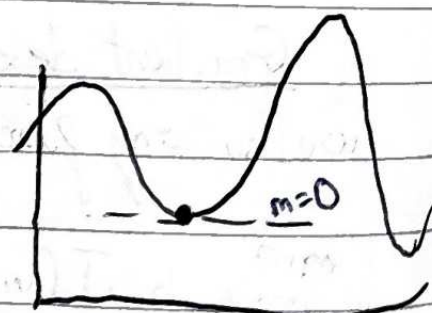
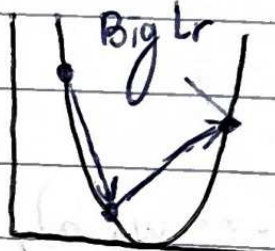
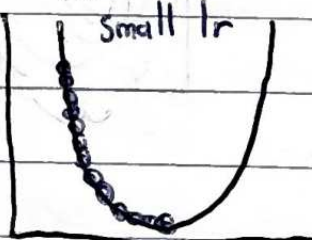
$$\begin{aligned} \text{tmp_w} &= w - \alpha \frac{\partial}{\partial w} J(w,b) \\ w &= \text{tmp_w} \\ \text{tmp_b} &= b - \alpha \frac{\partial}{\partial b} J(w,b) \\ b &= \text{tmp_b} \end{aligned}$$

LR

If too small $\rightarrow$ too slow

If too big $\rightarrow$ overshoots, fails to converge

dot representing
step or
compute



As you approach min,
the slope decreases so the
steps get smaller too.
 α is scaled by slope

$$w = w - \alpha \frac{d}{dw} J(w)$$

$$w = w - 0$$

no change needed :)

if you find yourself
at a local minimum
slope is 0

Using gradient descent on a linear model

Model: $f_{w,b}(x) = wx + b$

Cost function: $J(w,b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$

Gradient descent Algorithm:

Repeat until convergence {

$$\begin{aligned} w &= w - \alpha \left(\frac{d}{dw} J(w,b) \right) \longrightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)} \\ b &= b - \alpha \left(\frac{d}{db} J(w,b) \right) \longrightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \quad \text{no } x^{(i)} \text{ term} \end{aligned}$$

}

Calculating the slope

$$\frac{\partial}{\partial w} J(w,b) = \frac{d}{dw} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

$$= \frac{d}{dw} \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})^2$$

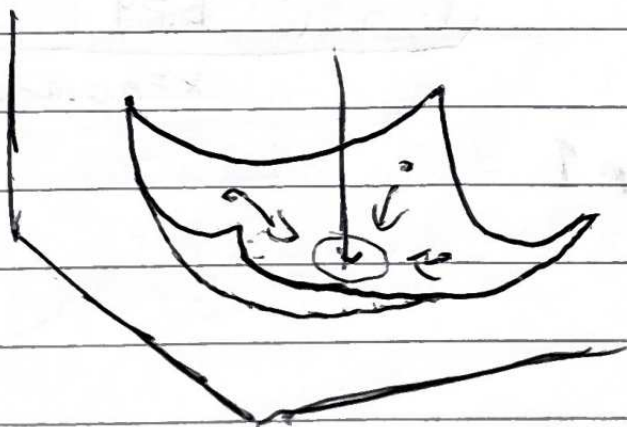
$$= \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cdot 2x^{(i)}$$

$$\frac{\partial}{\partial b} J(w,b) = \frac{d}{db} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{d}{db} \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})^2$$

$$= \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cdot 2$$

Squared error cost in linear models always gives a convex shape
so one local minima $\rightarrow$ global min

$J(w,b)$



Convergence :)

Multivariable Linear Regression (features)

	x_1	x_2	x_3	x_4	y
$[i]$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$[5]$	1	4	6	8	

$x_j = j^{\text{th}}$ feature

$n = \text{nr of features}$

$\vec{x}^{(i)} = \text{features of } i^{\text{th}} \text{ training example}$

$x^{(5)} = 1 \ 4 \ 6 \ 8 \rightarrow \text{row vector}$

$x_3^{(5)} = 6$

j / across / subscript

i / down / superscript

Example model

$$f_{w,b}(x) = \underbrace{w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4}_{\text{features}} + \underbrace{b}_{\text{base (guarantee independent of weights scale)}}$$

$$\vec{w} = [w_1, w_2, w_3, \dots, w_n] \quad \left. \begin{array}{l} \\ b \text{ is a nr} \end{array} \right\} \text{parameters}$$

$$\vec{x} = [x_1, x_2, x_3, \dots, x_n]$$

$\therefore$ simply can be written in terms of vectors!

Compact form

$$f_{\vec{w},b}(\vec{x}) = \underbrace{\vec{w} \cdot \vec{x}}_{\text{dot product}} + b$$

$$w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots + w_n x_n + b$$

Vectorization

Parameters and features

$$\vec{w} = [w_1, w_2, w_3] \quad n=3$$

b is a nr

$$\vec{x} = [x_1, x_2, x_3]$$

linear algebra: count from 1

Python (Numpy) $w[0]$

$$w = \text{np.array}([1.0, 2.5, -3.3])$$

$$b = 4$$

$$x = \text{np.array}([10, 20, 30])$$

$$\begin{array}{ccc} \downarrow & \downarrow & \downarrow \\ x[0] & x[1] & x[2] \end{array}$$

index 0
not 1

Without Vectorisation $n=100,000$

$$f(\vec{w}, b)(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$\text{range}(0, n) \rightarrow j = 0 \dots n-1$

$$f = w[0] * x[0] + \\ w[1] * x[1] + \\ w[2] * x[2] + b$$



$$F = 0$$

for j in range (n) :

$$F = F + w[j] * x[j]$$

$$F = F + b$$



With

$$f(\vec{w}, b)(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

$$f = \text{np.dot}(x, w) + b$$



Sequential vs
Parallel compute

Gradient descent

$$\vec{w} = (w_1, w_2, \dots, w_{16})$$

parameters

$$\vec{d} = (d_1, d_2, \dots, d_{16})$$

derivatives

Compute $w_j = w_j - 0.1 d_j$ for $j = 1 \dots 16$

Without Vectorization

$$w_1 = w_1 - 0.1 d_1$$

$$w_2 = w_2 - 0.1 d_2$$

$\vdots$

$$w_{16} = w_{16} - 0.1 d_{16}$$

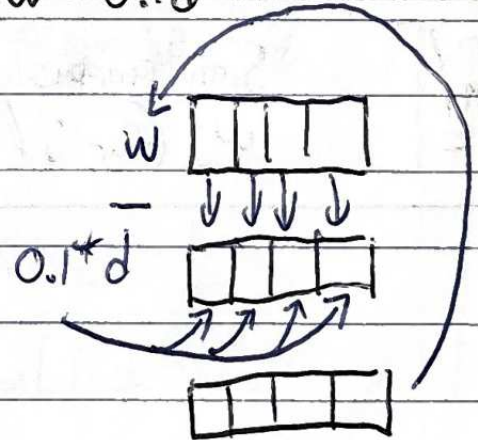
for j in range $(0, 16)$:

$$w[j] = w[j] - 0.1 * d[j]$$

With Vectorization

$$\vec{w} = \vec{w} - 0.1 \vec{d}$$

$$w = w - 0.1 * d$$



Repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

}

Derivatives for $n \geq 2$

1 feature $\left[w = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)} \right]$ old $\otimes$

for n features

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \quad \left(\text{for } j=1 \right)$$

$\hookrightarrow \partial / \partial \vec{w} J(\vec{w}, b)$

$$w_n = w_n - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_n^{(i)} \quad \left(\text{for } j=n \right)$$

1 feature $\left[b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \right]$ old $\otimes$

for vectorized

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

1 $\left[\text{simultaneously update } w, b \right]$ old $\otimes$

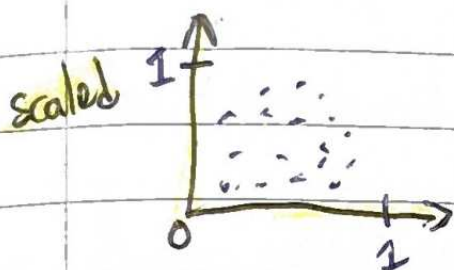
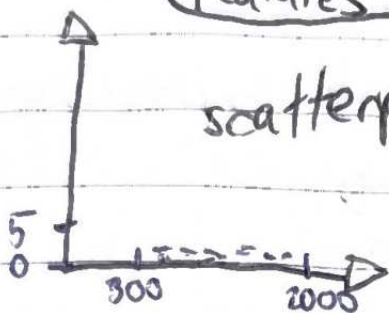
2
1
1 $\left[\text{Simultaneously Update } w_j \text{ (for } j=1, \dots, n) \text{ and } b \right]$

Practicalities $\rightarrow$ Linear regression

Feature Scaling

Features

scatterplot



Aim for $-1 \leq x \leq 1$ for each x_j
 $-3 \leq x \leq 3$
 $-0.3 \leq x \leq 0.3$ acceptable ranges

$0 \rightarrow 3$ okay
 $-2 \rightarrow 20.5$

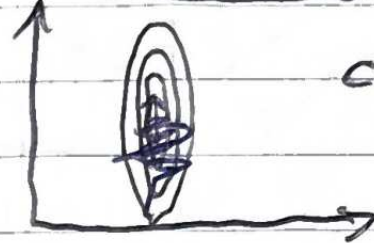
$-100 \rightarrow 100$
 too big

$-0.001 \rightarrow 0.001$
 too small

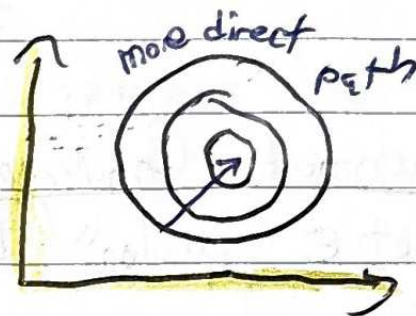
$98.6 \rightarrow 105$
 too big

rescaled

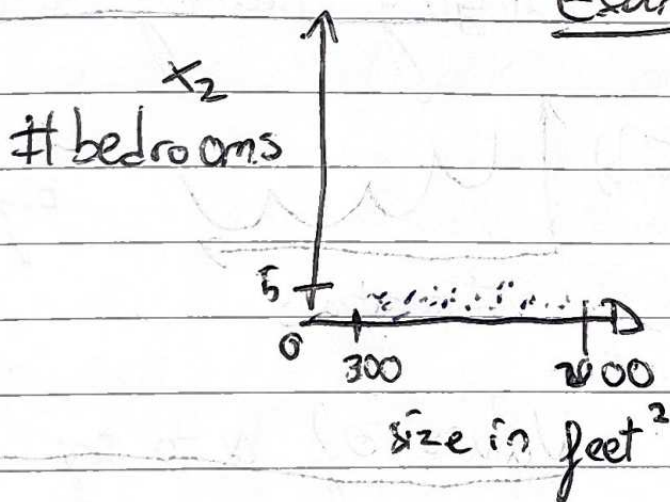
Parameters



contour plot
 $J(\vec{w}, b)$



Example



scale on maximum

$$300 \leq x \leq 2000 \quad \text{max}$$

$$x_1 \text{ scaled} = \frac{x_1}{2000}$$

$$0.15 \leq x_{\text{scaled}} \leq 1$$

Mean Normalization

take avg μ_1 and range

$$x_1 = \frac{x_1 - \mu_1}{2000 - 300}$$

$$-0.15 \leq x_1 \leq 0.82$$

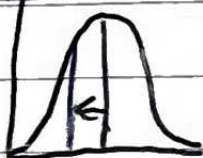
$$0 \leq x_2 \leq 5 \quad \text{max}$$

$$x_2 \text{ scaled} = x_2 / 5$$

$$0 \leq x_{2 \text{ scaled}} \leq 1$$

Z-score normalization

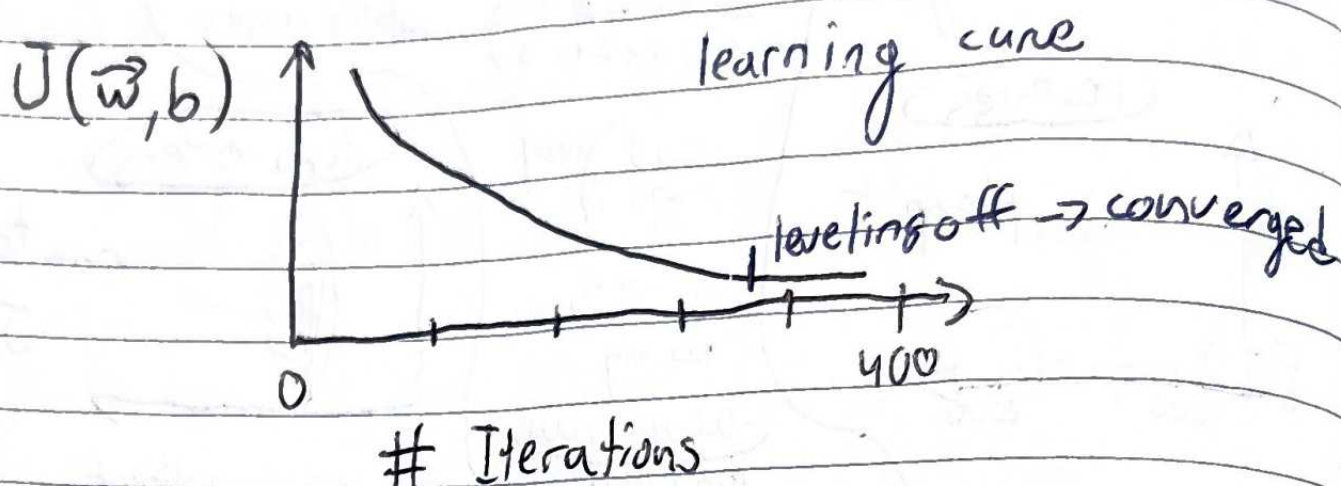
find σ_1 and μ_1



$$x_1 = \frac{x_1 - \mu_1}{\sigma_1}$$

$$-0.67 \leq x_1 \leq 3.1$$

Making sure Gradient descent is working



Automatic Convergence Test

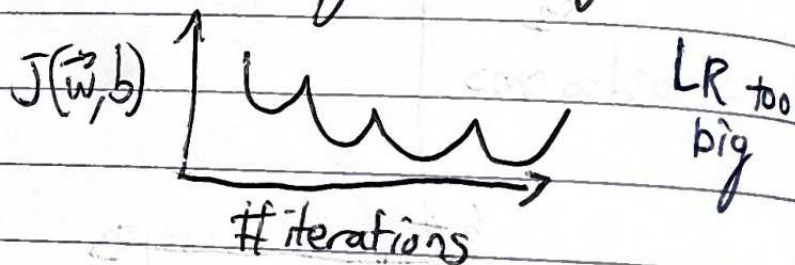
Let ϵ "epsilon" be 10^{-3}

If $J(\vec{w}, b)$ decreases $b \leq \epsilon$
in one iteration,
declare convergence.

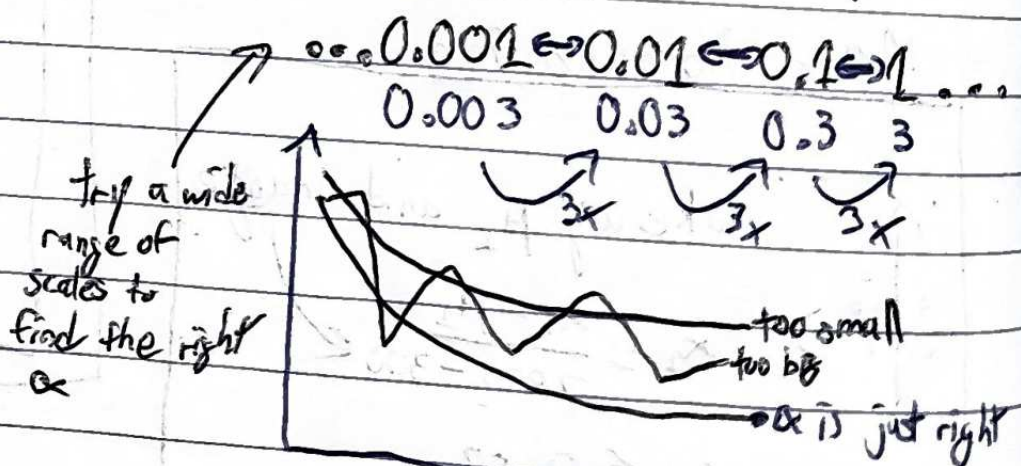
- with small enough α ,
 $J(\vec{w}, b)$ should decrease
on every iteration

- If too small $\rightarrow$ lot more
iterations to converge

Choosing Learning Rate



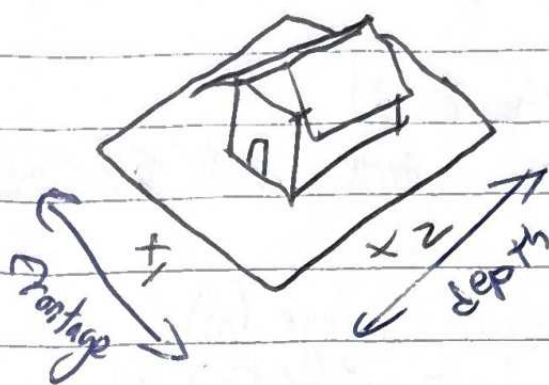
Values of α to try



Feature engineering

$$f_{\vec{w}, b}(\vec{x}) = \overbrace{w_1 x_1}^{\text{frontage}} + \overbrace{w_2 x_2}^{\text{depth}} + b \quad \text{Model}$$

Might be ok, but more effective could include area parameter



$$\text{area} = \text{frontage} \times \text{depth}$$

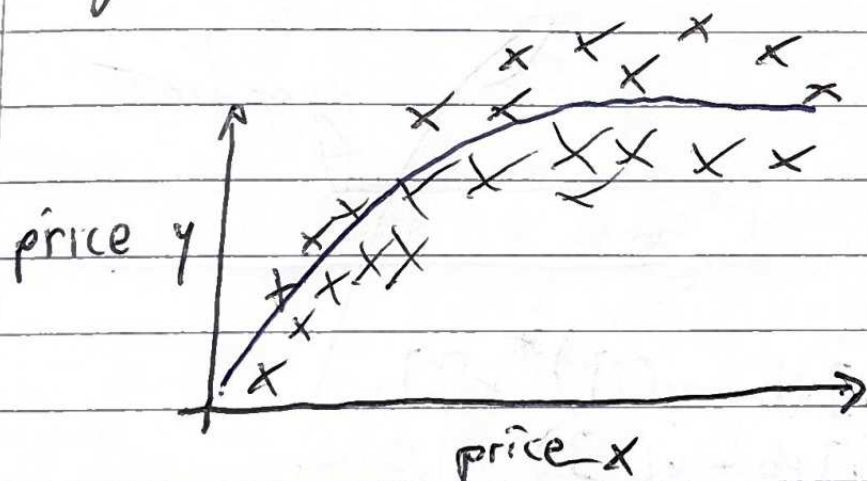
area may be more predictive of a price for a house $\rightarrow$ from intuition

$x_3 = x_1 x_2 \rightarrow$ new feature defined

$$f_{\vec{w}, b}(\vec{x}) = w_1 x_1 + w_2 x_2 + \underbrace{w_3 x_3}_{\text{area}} + b$$

Feature engineering is using intuition to design new features, by transforming or combining original features

Polynomial Regression - (linear regression and feature eng)



looking at this data a line doesn't really fit. so you may think of quadratics $\rightarrow$ but quadratics come back down? Cubic?

$\sqrt{x}$ could fit too



$$f_{\vec{w}, b}(x) = w_1 x + w_2 \sqrt{x} + b$$

$$f_{\vec{w}, b}(x) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$\uparrow$ size $\uparrow$ size $\uparrow$ size
 $1-10^3$ $1-10^6$ $1-10^9$

feature scaling!

Assignment

Nr of training examples

$$m = x.shape[0]$$

$$total_cost = 0$$

Cost

for i in range(m):

$$f_wb = w * x[i] + b$$

$$cost_i = (f_wb - y[i]) ** 2$$

$$total_cost = total_cost + cost_i$$

$$total_cost = (1 / (2 * m)) * total_cost$$

↑ Arguments are (x, y, w, b)

$$dj_dw = 0$$

$$dj_db = 0$$

gradient

for i in range(m):

$$tmp_dw = (w * x[i] + b - y[i]) * x[i]$$

$$tmp_db = (w * x[i] + b - y[i])$$

$$dj_dw = dj_dw + tmp_dw$$

$$dj_db = dj_db + tmp_db$$

$$dj_dw = dj_dw / m$$

$$dj_db = dj_db / m$$

Classification

yes, no $\rightarrow$ binary classification

class/category

no yes

0 1

false true

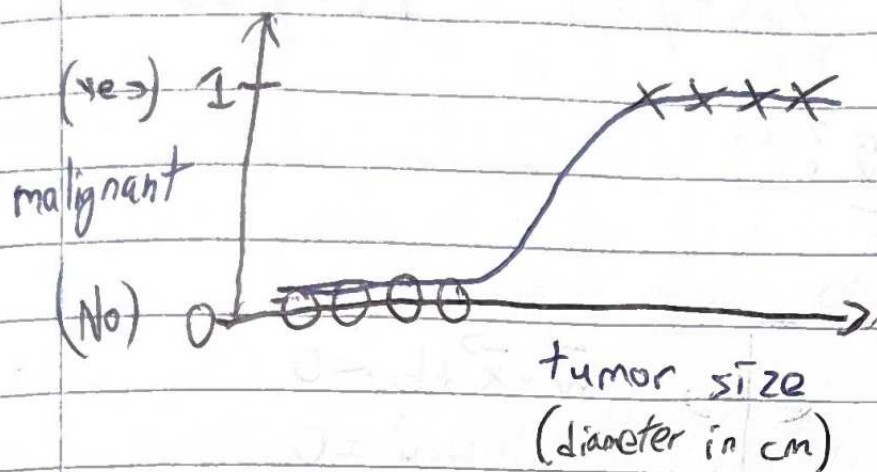
absence
negative

presence
positive

etc

Logistic Regression

Cancer



Sigmoid function

Want outputs between 0 and 1

$f_{\vec{w}, b}(\vec{x})$

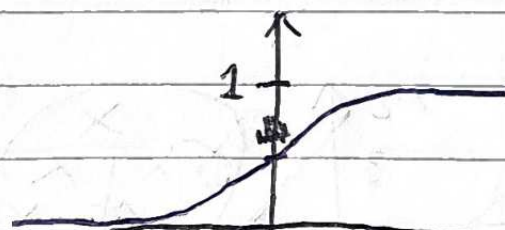
$$z = \vec{w} \cdot \vec{x} + b$$

$\downarrow$

z

$\downarrow$

$$g(z) = \frac{1}{1 + e^{-z}}$$



logistic function

$$g(z) = \frac{1}{1 + e^{-z}} \quad 0 < g(z) < 1$$

$$f_{\vec{w}, b}(\vec{x}) = g(\underbrace{\vec{w} \cdot \vec{x} + b}_z) = \frac{1}{1 + e^{-z}}$$

logistic regression

"probability" that class is 1 or "yes"

$$f_{\vec{w}, b}(\vec{x}) = 0.7$$

70% chance of
the tumor turning out
malignant

$\therefore$ 30% it is 0

or not malignant

w and b are
parameters affecting
this computation

$$f_{\vec{w}, b}(\vec{x}) = P(y=1 | \vec{x}; \vec{w}, b) = 1$$

Probability that y is 1, given input $\vec{x}$, parameters $\vec{w}, b$

$$P(y=0) + P(y=1) = 1$$

Decision Boundary

$$f_{\vec{w}, b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b)$$

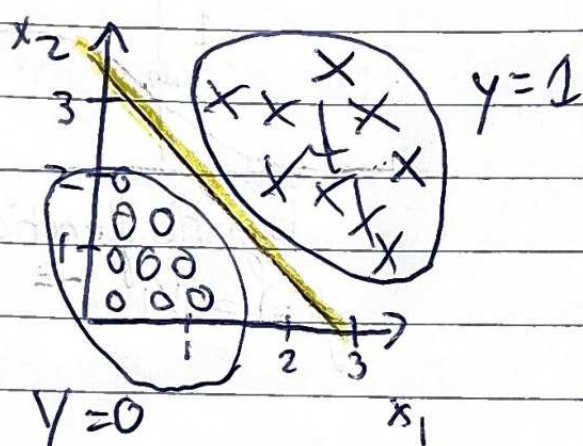


0 or 1? $\rightarrow$ threshold decides

is $f_{\vec{w}, b}(\vec{x}) \geq 0.5$?
 Yes: $\hat{y} = 1$ No: $\hat{y} = 0$

when is $f_{\vec{w}, b}(\vec{x}) \geq 0.5$?
 $g(z) \geq 0.5$
 $z \geq 0$

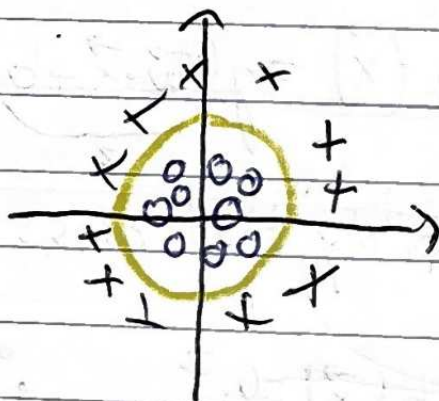
$$\begin{array}{c} \vec{w} \cdot \vec{x} + b \geq 0 \\ \hat{y} = 1 \end{array} \quad \longleftrightarrow \quad \begin{array}{c} \vec{w} \cdot \vec{x} + b < 0 \\ \hat{y} = 0 \end{array}$$



Decision boundary is @
 $z = \vec{w} \cdot \vec{x} + b = 0$
 derived from sigmoid function

Non-linear decision boundaries

Just like with linear regression,
 we can "engineer" the function
 let z or $f_{\vec{w}, b}$ be circular in its
 shape



$$g(\underbrace{w_1 x_1^2 + w_2 x_2^2 + b}_z) \quad \rightarrow \quad \begin{array}{l} z = x_1^2 + x_2^2 - 1 = 0 \\ x_1^2 + x_2^2 = 1 \end{array}$$

you can fit complex features with
 higher and higher order polynomials

$$\begin{array}{c} x_1^2 + x_2^2 \geq 1 \\ \hat{y} = 1 \end{array} \quad \longleftrightarrow \quad \begin{array}{c} x_1^2 + x_2^2 < 1 \\ \hat{y} = 0 \end{array}$$

$\hat{y} = 1$

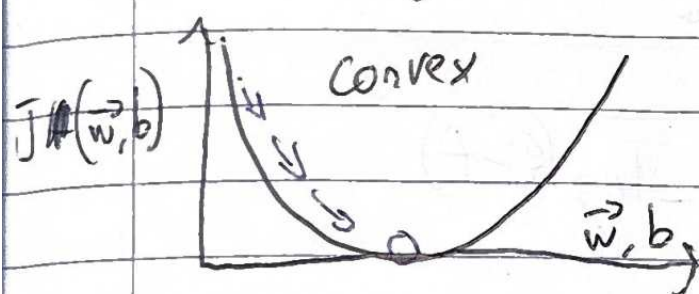
Cost function for logistic regression

Training set

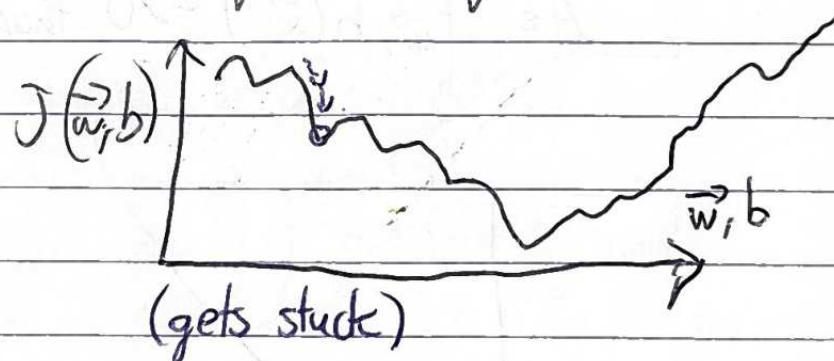
	x_1 Tumor size	...	x_n patient's age	y malignant?	
$i=1$	10		52	1	$i = 1, \dots, m$ $j = 1, \dots, n \in \text{features}$ training examples target y is 0 or 1
$\vdots$	2		73	0	
$\vdots$	5		55	0	
	12		49	1	
$i=m$	...		...	...	

Squared error cost $\rightarrow$ Why not?

linear regression



logistic regression

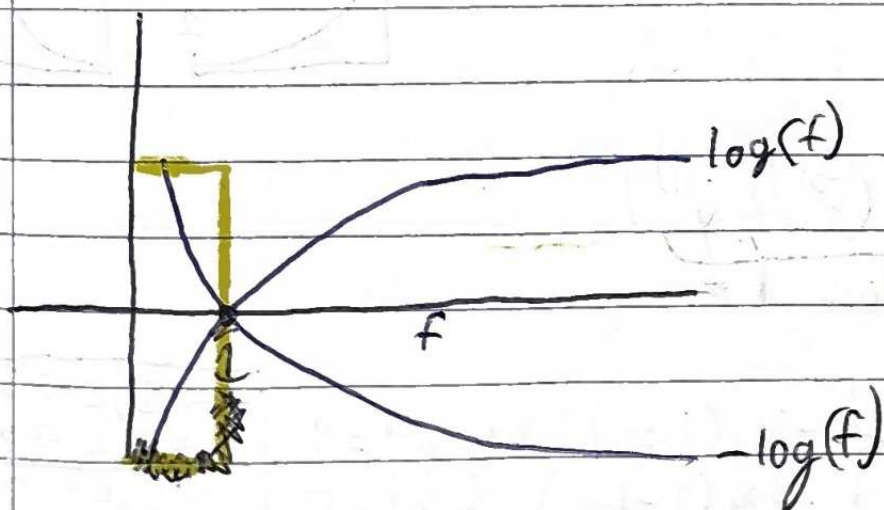


Logistic loss function

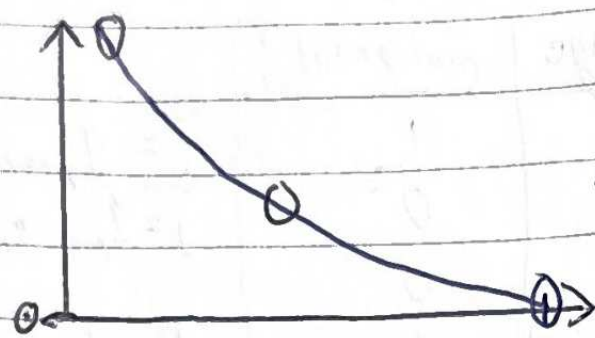
$$L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w}, b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

yes? no?

f is output of graph $\therefore$ it is always between 0 and 1 (logistic regression)



Zooming in and explain



$-\log(f)$

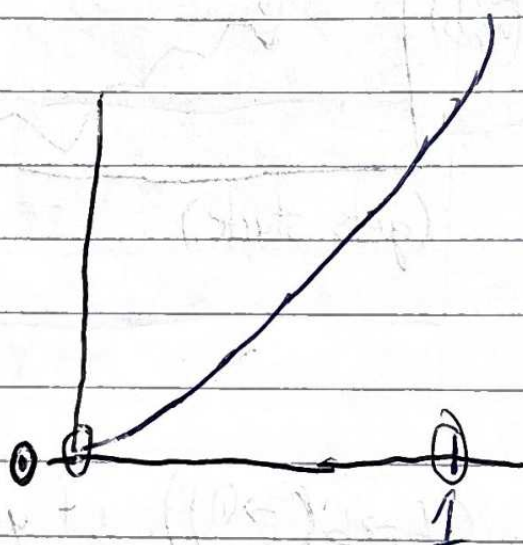
As f approaches 1, loss $\rightarrow 0$
as f approaches 0, loss $\rightarrow \infty$

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})$$

$-\log(1) = 0$ if $y^{(i)} = 1$
As $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 1$ then loss $\rightarrow 0$

loss is lowest
when $f_{\vec{w},b}(\vec{x}^{(i)})$
predicts close to
true label $y^{(i)}$

As $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 0$ then loss $\rightarrow \infty$



$-\log(1-f)$

As f approaches 0 then loss $\rightarrow 0$

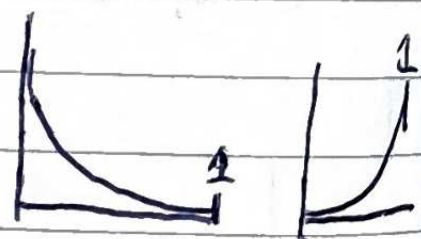
As f approaches 1 then loss $\rightarrow \infty$

To summarize

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(\underbrace{f_{\vec{w},b}(\vec{x}^{(i)})}_{\text{loss}}, \underbrace{y^{(i)}}_{*})$$

$\left\{ \begin{array}{ll} -\log(\text{loss}) & \text{if } y^{(i)} = 1 \\ -\log(1-\text{loss}) & \text{if } y^{(i)} = 0 \end{array} \right.$

$f \rightarrow 1, L \rightarrow 0$
$f \rightarrow 0, L \rightarrow \infty$
$f \rightarrow 1, L \rightarrow \infty$
$f \rightarrow 0, L \rightarrow 0$



Loss function $\rightarrow$ simplified

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$$

Gates

factored out of both negative logs

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))]$$

Derived from Statistics $\rightarrow$ Maximum likelihood $\rightarrow$ Convex too

Gradient descent implementation

Ofc repeat $\left[\begin{array}{l} w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \\ b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \end{array} \right]$ as per usual

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

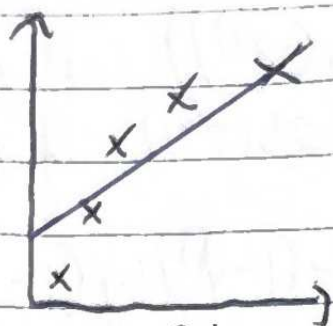
same as linear regression but the function itself changed

linear regression $\rightarrow f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

logistic regression $\rightarrow f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$

Same concepts $\rightarrow$ Monitor Gradient Descent (learning curve)
 $\rightarrow$ Vectorized implementation
 $\rightarrow$ Feature scaling

Overfitting



- underfit
- does not fit the training set well
- high bias

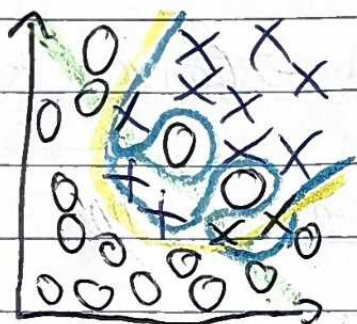


- fits training set pretty well
- generalization just right



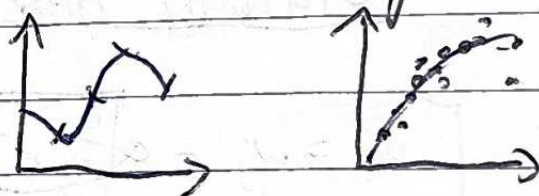
- fits training set extremely well
- overfits
- high variance

in Classification



- overfit
- underfit
- just right

Tackling overfitting



Collect more training examples
or

Select features to include/exclude

[useful features could be lost!]

Regularisation

you can manually set a weight of a feature to 0 and let it relearn or reduce size

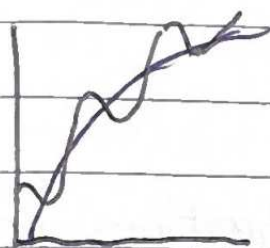
$$f(x) = 28x - 385x^2 + 39x^3 - \underbrace{174x^4}_0 + 100$$

Penalising

You can modify the cost function to minimize for example w_3 and w_4

$$\min_{\vec{w}, b} \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + 1000 w_3^2 + 1000 w_4^2$$

for some $w_1 x + w_2 x^2 + \underbrace{w_3 x^3}_{\approx 0} + \underbrace{w_4 x^4}_{\approx 0} + b$



1000 is in the way, forced to modify "w" to minimize, diminishing the contributions of 3rd and 4th features to the prediction.

if you penalise all features $\rightarrow$ you create a simpler model, less prone to overfitting

$\rightarrow$ Implement this with the following:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

$\lambda > 0$

~~lambda~~ lambda

the regularisation parameter

Regularised linear regression

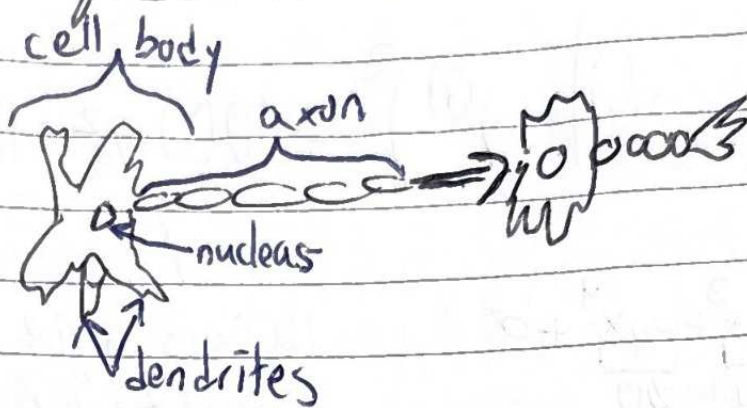
The derivative of $J(\vec{w}, b)$ stays the same just add derived lambda term:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j$$

Same for ~~lambda~~ etc, etc $\rightarrow \partial/\partial b$ doesn't really get affected
(logistic regression)

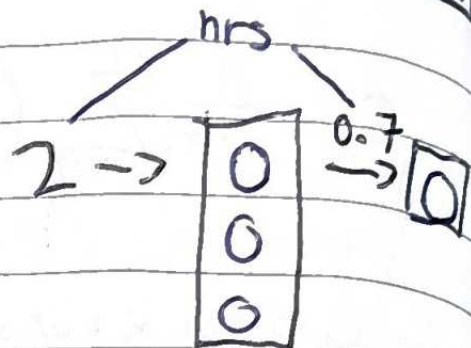
Neural Networks

Biological Neuron

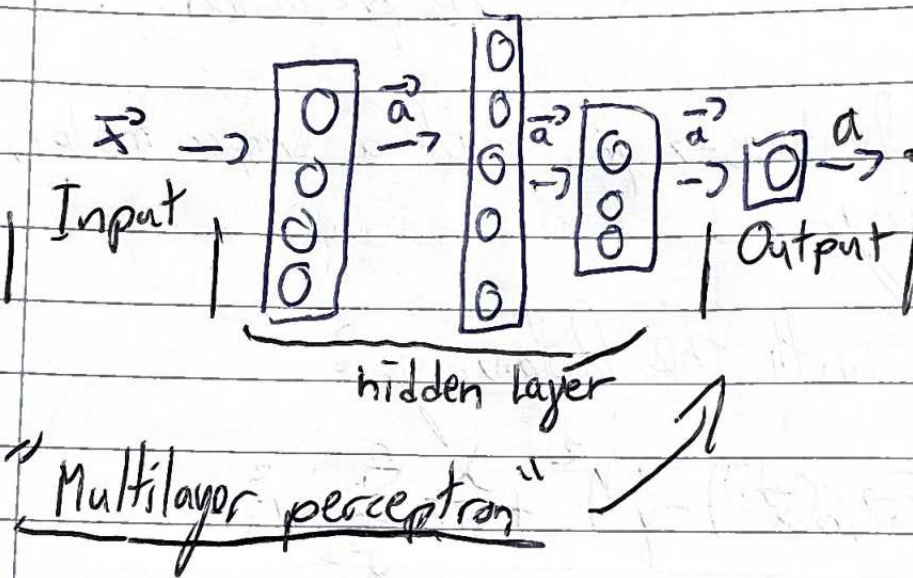


Inputs $\rightarrow$ Output

Simplified Math model

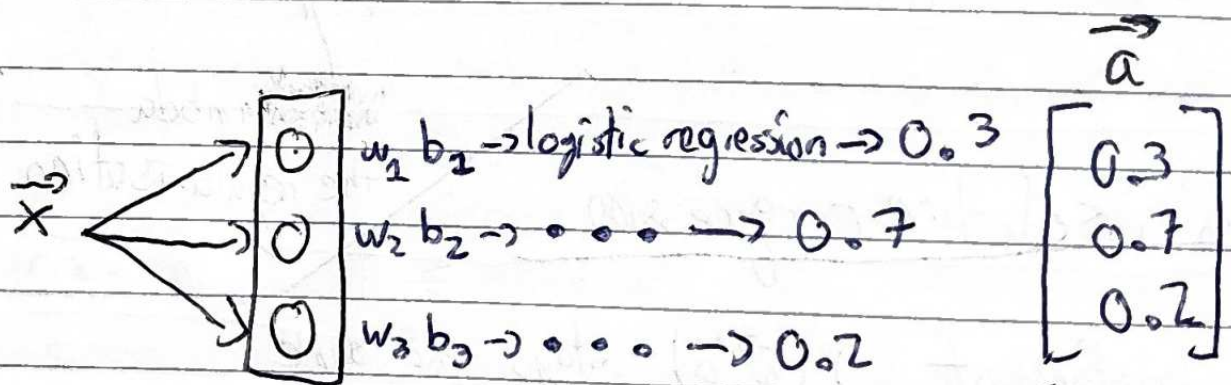


activations are higher level features



you can add bias to make it yes or no rather than a probability

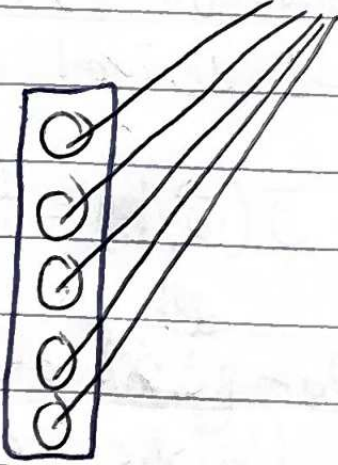
is $a^{[4]} \geq 0.5$?
 $\hat{y} = 1$ $\hat{y} = 0$



Superscript $[1]$ for layer 1,
 $[0]$ for input, $[2]$ for layer 2 etc...

Examples

- $a^{[1]}$
- $w_1^{[1]}$
- $w_2^{[1]}$
- $b_3^{[1]}$



General Notation

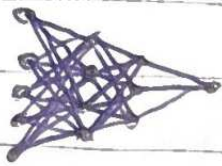
Activation value
of layer l , unit (neuron)
 j

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$



Sigmoid is the activation function here $g()$

forward propagation (inference)



backward propagation (learning)

Tensorflow - (simple code for a simple multilayer perceptron inference)

```
x = np.array([[200.0, 17.0]])
```

```
layer_1 = Dense(units=3, activation='sigmoid')
```

```
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')
```

```
a2 = layer_2(a1)
```

```
if a2 > 0.5:
```

```
    yhat = 1
```

```
else:
```

```
    yhat = 0
```

$\vec{x}$

0 0 0

$\vec{a}^{[1]}$

0

$a^{[2]}$

Y/N?

$a^{[2]} > 0.5$

$\hat{y} = 1$

$\hat{y} = 0$

Numpy arrays

```
x = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

matrix

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

Numpy Arrays

matrices $\left[\begin{array}{l} x = \text{np.array}([200, 17]) \quad [200 \quad 17] \quad 1 \times 2 \\ x = \text{np.array}([200], [17]) \quad \begin{bmatrix} 200 \\ 17 \end{bmatrix} \quad 2 \times 1 \end{array} \right.$

$x = \text{np.array}([200, 17]) \rightarrow$ 1D vector aka list

Different way of implementing a NN in Tensorflow

layer_1 = Dense(units=3, activation="sigmoid")

layer_2 = Dense(units=1, activation="sigmoid")

model = Sequential([layer_1, layer_2])



4x2 $x = \text{np.array}([200.0, 17.0], [120.0, 5.0], [425.0, 20.0], [212.0, 18.0]))$

			Y
	200	17	1
	120	5	0
	425	20	0
	212	18	1

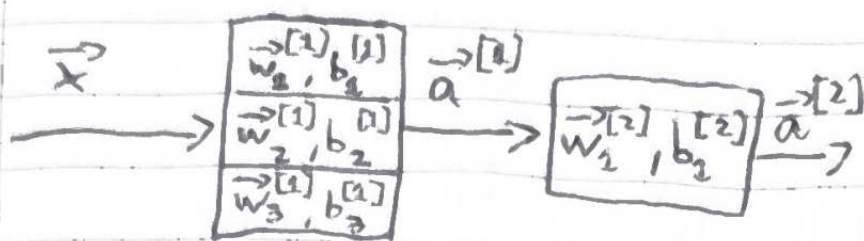
targets $y = \text{np.array}([1, 0, 0, 1])$

model.compile(...)

model.fit(x, y)

model.predict(x_new) $\in$ inference

forward prop



$x = \text{np.array}([200, 17])$

$$a_1^{[1]} = g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]}) / a_2^{[1]} = g(\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]}) / a_3^{[1]} = g(\vec{w}_3^{[1]} \cdot \vec{x} + b_3^{[1]})$$

$w1_1 = \text{np.array}([1, 2])$

$b1_1 = \text{np.array}([-1])$

$z1_1 = \text{np.dot}(w1_1, x) + b1_1$ dot product btw

$a1_1 = \text{sigmoid}(z1_1)$

~~all~~

$w1_2 = \text{np.array}([-3, 4])$

$b1_2 = \text{np.array}([1])$

$z1_2 = \text{np.dot}(w1_2, x) + b1_2$

$a1_2 = \text{sigmoid}(z1_2)$

$w1_3 = \text{np.array}([5, -6])$

$b1_3 = \text{np.array}([2])$

$z1_3 = \text{np.dot}(w1_3, x) + b1_3$

$a1_3 = \text{sigmoid}(z1_3)$

$a1 = \text{np.array}([a1_1, a1_2, a1_3])$

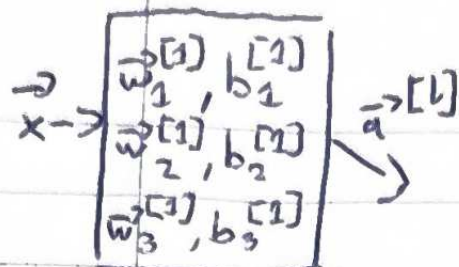
$$a_1^{[2]} = g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]})$$

$w2_1 = \text{np.array}([-7, 8, 9])$

$b2_1 = \text{np.array}([3])$

$z2_1 = \text{np.dot}(w2_1, a1) + b2_1$

$a2_1 = \text{sigmoid}(z2_1)$



General Implementation of forward prop

```
def dense(a_in, W, b):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:, j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```

$$\vec{w}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\vec{w}_2 = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$$

$$\vec{w}_3 = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$$

$$b_1 = -1$$

$$b_2 = 1$$

$$b_3 = 2$$

$$\vec{a} = \vec{x}$$

```
def sequential(x):
    a1 = dense(x, W1, b1)
    a2 = dense(a1, W2, b2)
    a3 = dense(a2, W3, b3)
    a4 = dense(a3, W4, b4)
    f_x = a4
    return f_x
```

∴ We can feel like this...

```
W = np.array([[1, -3, 5],
              [2, 4, -6]])
```

```
b = np.array([-1, 1, 2])
```

```
a_in = np.array([-2, 4])
```

Training a NN in Tensorflow

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
```

```
Dense(units=25, activation='sigmoid'),
Dense(units=15, activation='sigmoid'),
Dense(units=1, activation='sigmoid'),
```

])

```
from tensorflow.keras.losses import BinaryCrossentropy
```

```
model.compile(loss=BinaryCrossentropy())
```

```
model.fit(X, Y, epochs=100)
```

↳ steps of gradient descent

(1)

(2)

(3)

Model Training Steps

IF

① specify how to compute output given input x and parameters w, b (define model) $f_{\vec{w}, b}(\vec{x}) = ?$	logistic regression $z = \text{np.dot}(w, x) + b$ $f_x = 1 / (1 + \text{np.exp}(-z))$	Neural Network Model = Sequential([Dense(...) Dense(...) Dense(...) 1])
② specify loss and cost eg. $L(f_{\vec{w}, b}(\vec{x}), y)$ $J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)})$	logistic loss $\text{loss} = -y * \text{np.log}(f_x) - (1-y) * \text{np.log}(1-f_x)$ $w = w - \text{alpha} * dj_dw$ $b = b - \text{alpha} * dj_db$	binary cross entropy model.compile(loss = BinaryCrossEntropy())
③ Train on data to minimize $J(\vec{w}, b)$		model.fit(X, y, epochs=100)

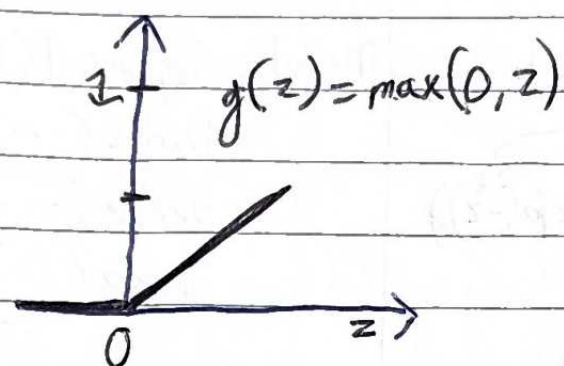
'BinaryCrossEntropy' is logistic loss !!
'Mean Squared Error' is MSE

TF

model.fit is just backprop calculating all the partial derivatives for gradient descent

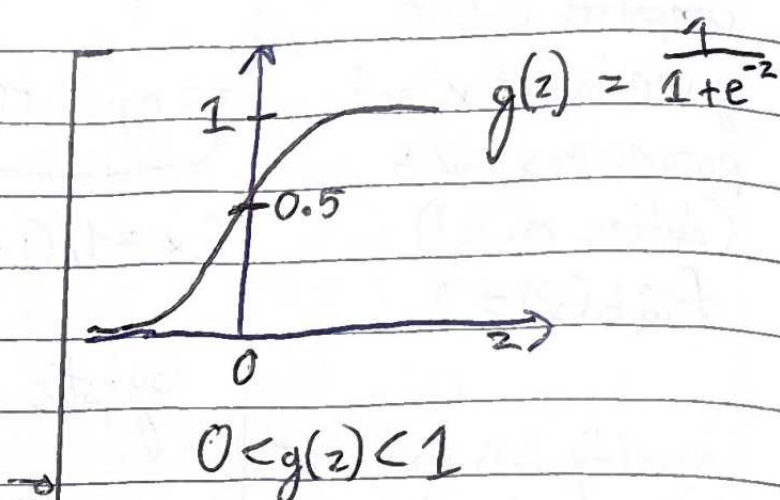
Alternatives to Sigmoid

Activation functions



if $z < 0$ if $z \geq 0$
 $g(z)$ is 0 $g(z) = z$

ReLU (Rectified Linear Unit)



Sigmoid

No activation function may be referred to as $g(z) = z$ aka linear activation function

Choosing activation $\rightarrow$ Output layer

If working on a classification problem, choose sigmoid on output layer

(Y/N)

$y = 0/1$

If working on regression or correlation, choose Linear activation function for output layer

$\nearrow$

$y = +/-$

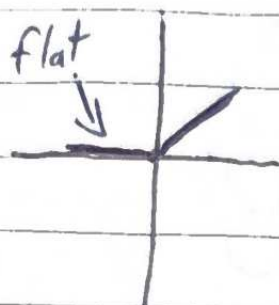
But if working on tasks that may not take on negative values for any instance eg. House prices, use ReLU

$y = 0 \text{ or } +$

Choosing activation $\rightarrow$ Hidden layers

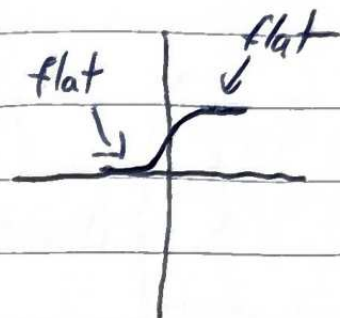
ReLU is most popular!

ReLU is faster than Sigmoid to compute $\rightarrow$ you just take "max"



ReLU also has less instances of "flat".

When you have a function that is flat in a lot of places, gradient descent is slow.



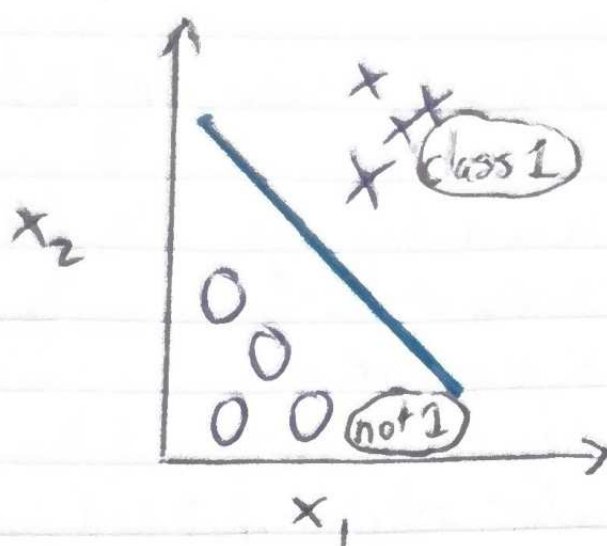
Gradient descent minimizes $J(w, b)$ but activation functions take up a piece of that pie so that is why training slows down

If we don't use activation functions, there's no point to a neural network, no different to a regression model

Think of ReLU as switching a function off until needed.
ReLU activations allow a model to stitch together linear segments to model complex non-linear functions!!!

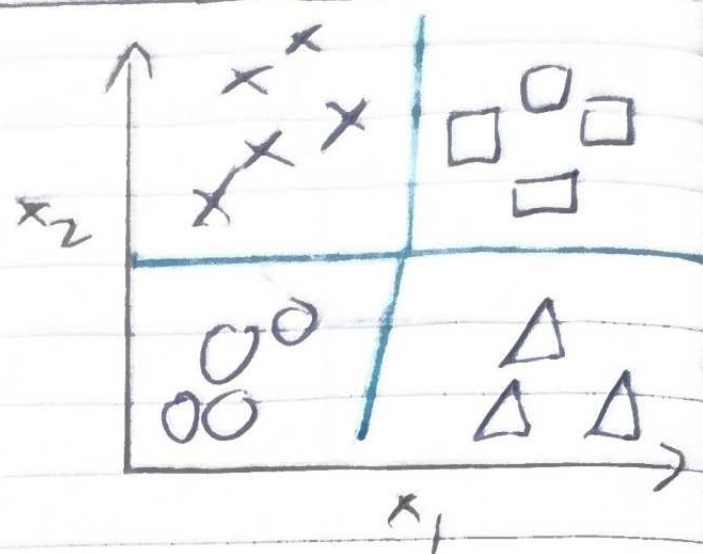
Multiclass Classification

target y can take on more than 2 possible values



$$P(y=1|\vec{x})$$

$$P(y=0|\vec{x})$$



$$P(y=1|\vec{x})$$

$$P(y=2|\vec{x})$$

$$P(y=3|\vec{x})$$

$$P(y=4|\vec{x})$$

Logistic Regression

(2 possible output values)

$$z = \vec{w} \cdot \vec{x} + b$$

$$X \quad a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x}) \quad 0.71$$

$$O \quad a_2 = 1 - a_1 = P(y=0|\vec{x}) \quad 0.29$$

Softmax Regression (N possible outputs)

$$z_j = \vec{w}_j \cdot \vec{x} + b_j, \quad j=1, \dots, N$$

parameters $w_1, w_2, \dots, w_N$
 $b_1, b_2, \dots, b_N$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j|\vec{x})$$

Softmax Reg Example for $N=4$ $y=1,2,3,4$

$$X \quad z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$a_1 = \frac{e^{z_1}}{(e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4})} = P(y=1|\vec{x})$$

$$O \quad z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

$$a_2 = \frac{e^{z_2}}{(e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4})} = P(y=2|\vec{x})$$

$$\square \quad z_3 = \vec{w}_3 \cdot \vec{x} + b_3$$

$$a_3 = \frac{e^{z_3}}{(e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4})} = P(y=3|\vec{x})$$

$$\triangle \quad z_4 = \vec{w}_4 \cdot \vec{x} + b_4$$

$$a_4 = \frac{e^{z_4}}{(e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4})} = P(y=4|\vec{x})$$

Logistic regression

$$z = \vec{w} \cdot \vec{x} + b$$

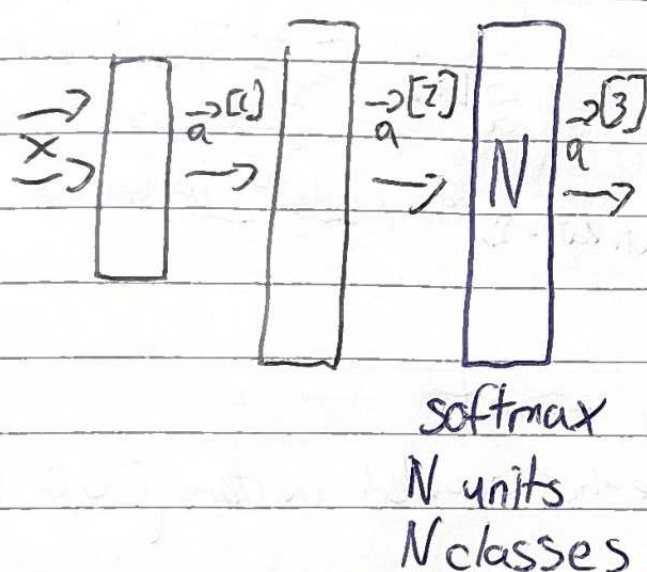
$$a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x})$$

$$a_2 = 1 - a_1 = P(y=0|\vec{x})$$

$$\text{loss} = -y \log a_1 - (1-y) \log (1-a_1)$$

if $y=1$ if $y=0$

$$J(\vec{w}, b) = \text{avg loss}$$



Change to this

`model.compile(loss=Sparse... (from_logits=True))`

Numerically more accurate

Change this too

`Dense(units=10, activation='linear')`

and add this block

`logit = model(X)`

`f_x = tf.nn.softmax(logit)`

Cost

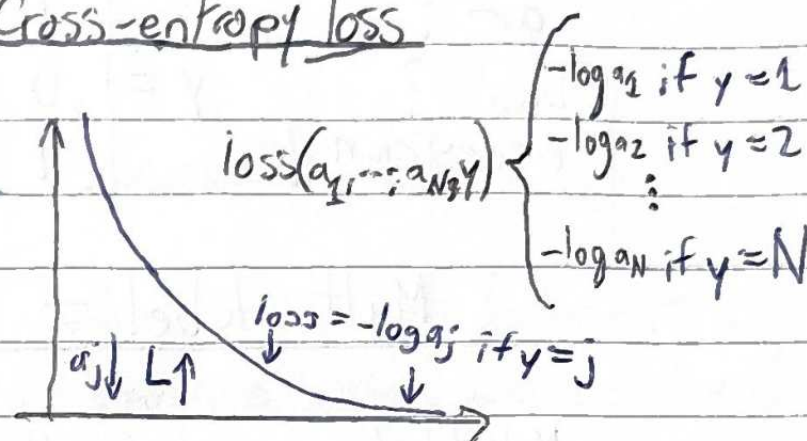
Softmax regression

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y=1|\vec{x})$$

$$\vdots$$

$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y=N|\vec{x})$$

Cross-entropy loss



TF

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='softmax')
])
```

```
from tensorflow.keras.losses import
SparseCategoricalCrossentropy
model.compile(loss=())
model.fit(X, Y, epochs=100)
```

computers store a finite precision
approx vs actual mathematical value
so the above code isn't too
great

$$\text{loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y=1 \\ -\log a_{10} & \text{if } y=10 \end{cases}$$

vs More accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y=1 \\ \vdots \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y=10 \end{cases}$$

Multi-label Classification

where you ask 3 questions about data
for example:

in a given image is there a?

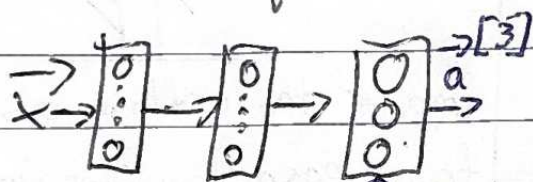
car?
bus?
pedestrian?

$$y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

you may train 3 separate
models that answer each Q
individually

Multi-label $\neq$ Multi class

Multilabel can target all 3 y-values via sigmoids like such:



sigmoid activations

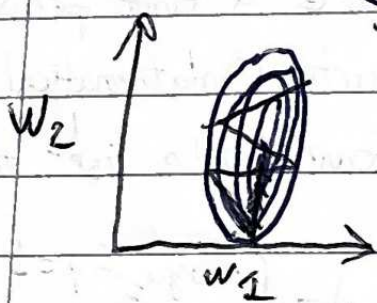
$$a^{[3]} = \begin{bmatrix} a_1^{[3]} \\ a_2^{[3]} \\ a_3^{[3]} \end{bmatrix} \begin{matrix} \text{car} \\ \text{bus} \\ \text{pedestrian} \end{matrix}$$

Sparse Categorical Cross Entropy $\rightarrow$ int
Categorical Cross Entropy $\rightarrow$ one-hot encoded vectors $[0, 0, 1, 0]$

Advanced Optimisation

Gradient Descent

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$



Whether the learning
rate is too big
or too small, the
"Adam" algorithm adjusts
it to optimum auto-
matically

too big $\rightarrow$ smaller α
too small $\rightarrow$ bigger α

Adam: Adaptive Moment estimation

What does it do?

if w_j (or b) keeps moving in the same
direction, increase α_j

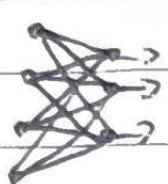
if they keep oscillating,
reduce α_j

Implementing Ham in TF

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

Additional Layers

Dense



Each neuron output is a function of all the activation

outputs of the previous layer.

$$\vec{a}_1 \cdot \vec{z} = g(\vec{w}_1 \cdot \vec{z} + b_1)$$

Convolutional

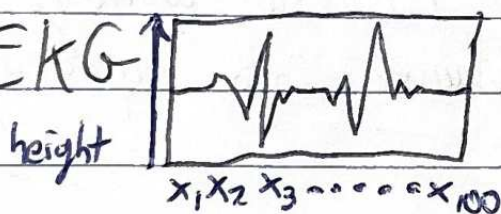
Each neuron only looks at a part of the previous layer's outputs (Regions)

Why?

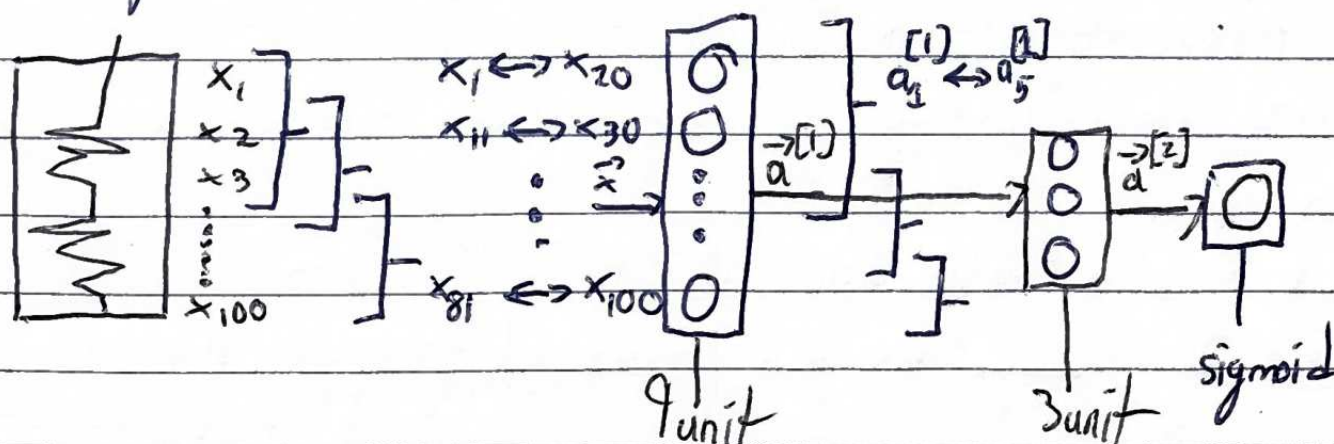
- Faster computation
- Need less train data (less prone to overfitting)

Convolutional NN

Example: EKG



Given this EKG signal time series, classify whether this patient has a heart disease or some diagnosable heart conditions



Debugging a Learning Algorithm

You've implemented regularized linear regression on housing prices

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

But it makes unacceptably large errors in predictions. What do you try next?

- get more training examples → good for high variance
- try smaller sets of features → good for high variance
- try getting additional features → fixes high bias
- try adding polynomial features ($x_1^2, x_2^2, x_1 x_2$, etc) → fixes high bias
- try decreasing λ → fixes high bias
- try increasing λ → fixes high variance

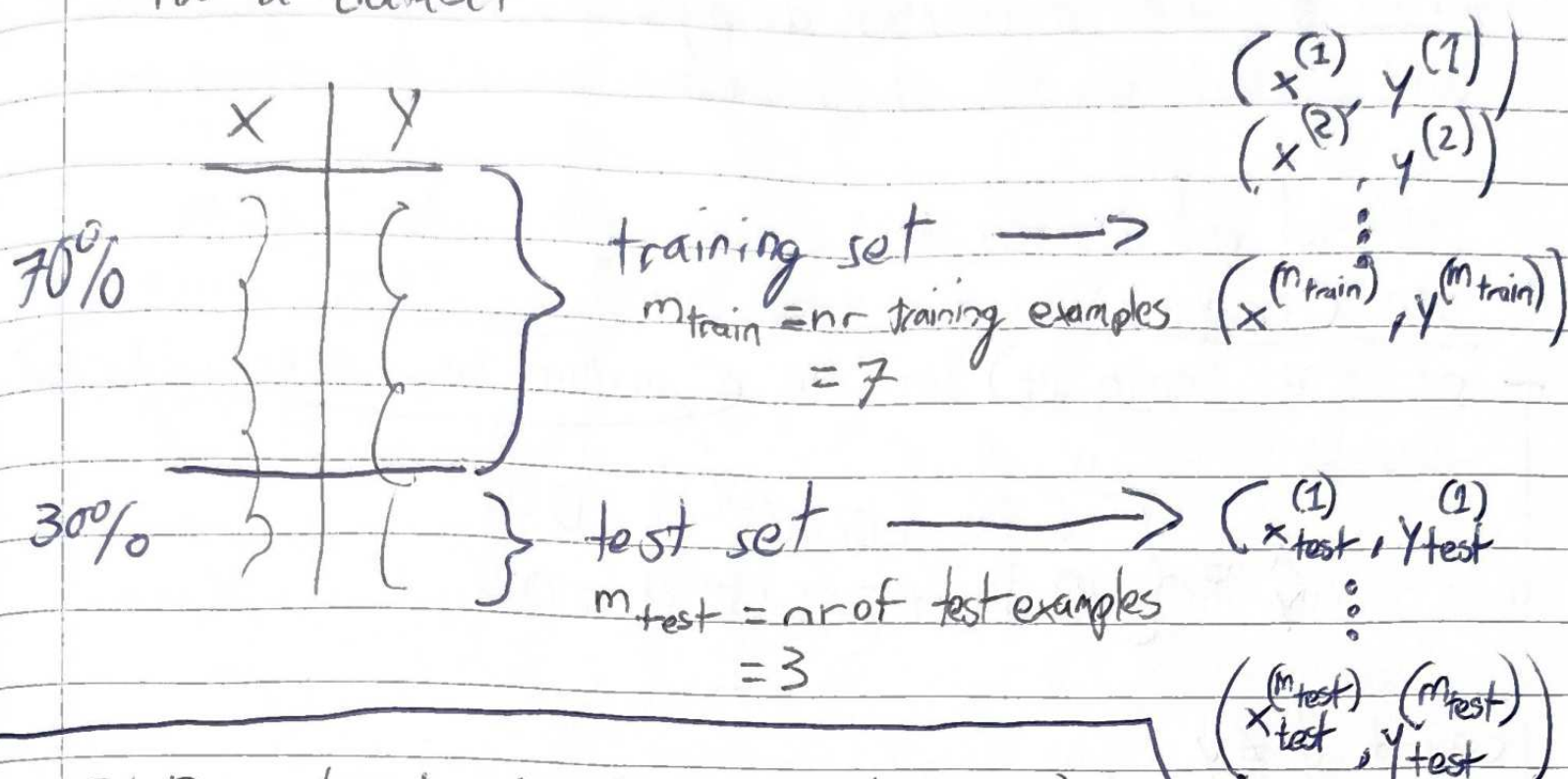
ML Diagnostics

Gain insight into what is/isn't working with a learning algorithm, to gain guidance into improving its performance

Diagnostics can take time to implement but doing so can be a very good use of your time.

Evaluating model

For a dataset

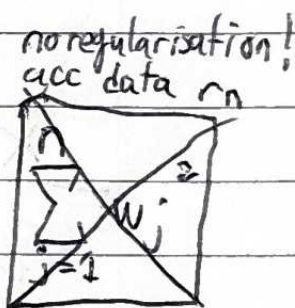


Fit Parameters by minimizing cost function $J(\vec{w}, b)$

$$\rightarrow J(\vec{w}, b) = \left[\frac{1}{2m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m_{\text{train}}} \sum_{j=1}^n w_j^2 \right]$$

Compute test error

$$\rightarrow J_{\text{test}}(\vec{w}, b) = \frac{1}{2m_{\text{test}}} \left[\sum_{i=1}^{m_{\text{test}}} (f_{\vec{w}, b}(\vec{x}_{\text{test}}^{(i)}) - y_{\text{test}}^{(i)})^2 \right]$$



Compute training error

$$\rightarrow J_{\text{train}}(\vec{w}, b) = \frac{1}{2m_{\text{train}}} \left[\sum_{i=1}^{m_{\text{train}}} (f_{\vec{w}, b}(\vec{x}_{\text{train}}^{(i)}) - y_{\text{train}}^{(i)})^2 \right]$$

Is tests generalisation on linear regression

Train/Test Procedure for classification

~~Train~~ Same principles apply. Get rid of regularisation and substitute for the formulas $\rightarrow$ except for training etc.

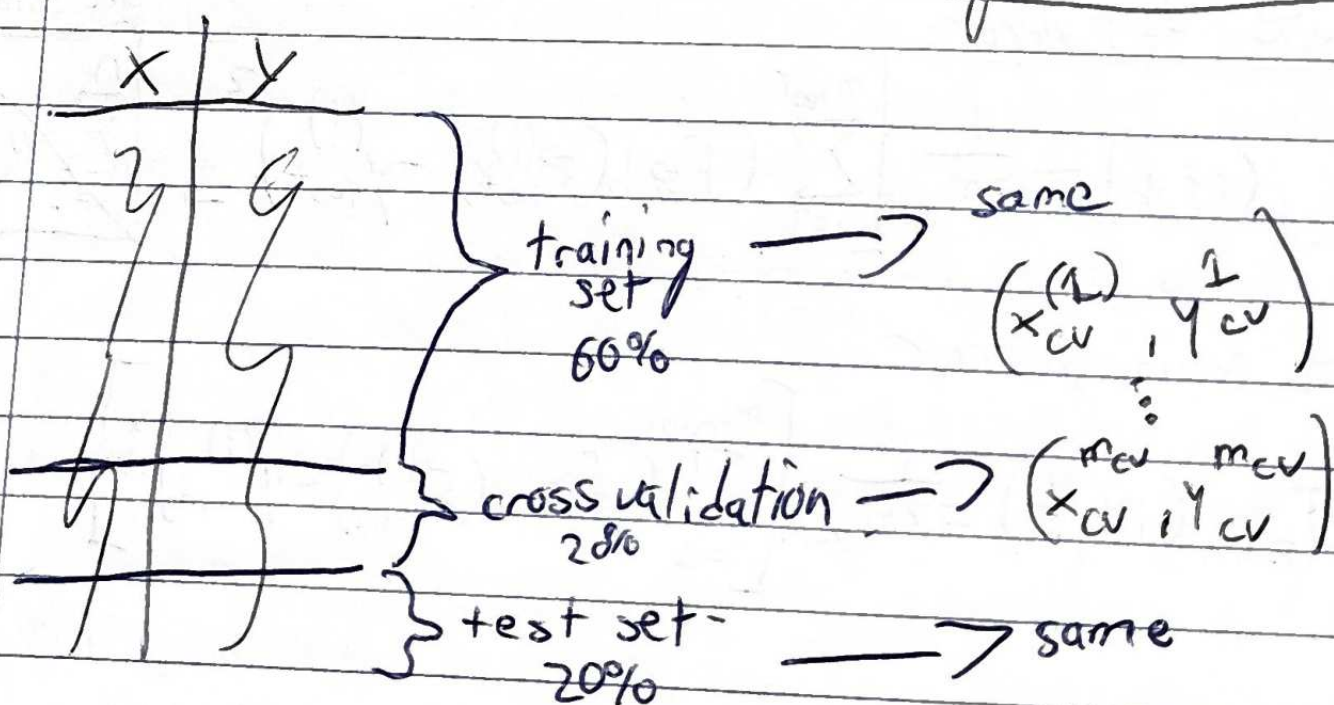
Instead of logistic loss you may also take the fraction of the test set and the fraction of the train set that the algorithm has misclassified

$$\hat{y} = \begin{cases} 1 & \text{if } f_{\vec{w}, b}(\vec{x}^{(i)}) \geq 0.5 \\ 0 & \text{if } f_{\vec{w}, b}(\vec{x}^{(i)}) < 0.5 \end{cases}$$

count $\hat{y} \neq y$

$J_{\text{test}}(\vec{w}, b)$ is the fraction of the test set that has been misclassified.
 $J_{\text{train}}(\vec{w}, b)$ is the fraction of the train set that has been misclassified.

Model Selection and training/cross-validation/tests



cross val across diff architectures to find best one.
Training set to train
cv to optimise architecture/order of polynomials
test set only once for the final $\rightarrow$ tests generalisation

Establishing a baseline level of performance

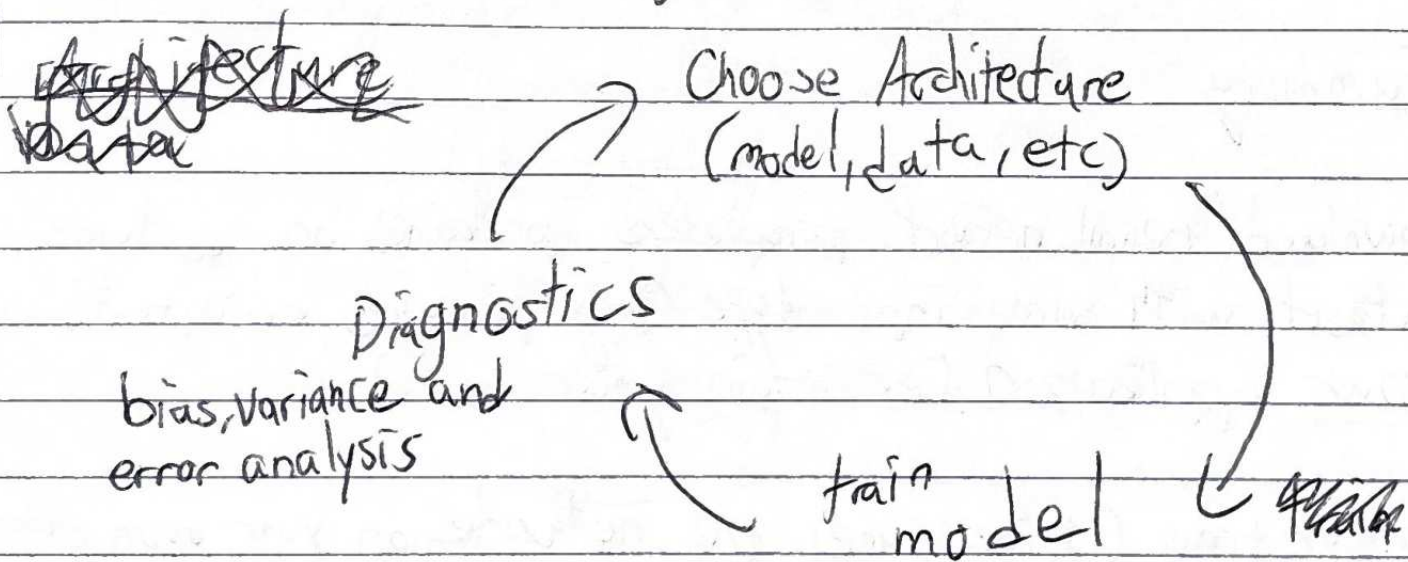
What is the level of error you can reasonably hope to get to?

- Human level performance
- Competing algorithms performance
- Guess based on experience

Baseline performance	10.5%	↕ bias measure
Training error (J_{train})	10.8%	
Cross validation err (J_w)	14.8%	↕ variance

Dense (kernel_regularizer = L2(λ)) $\rightarrow$ Regularisation in T

Machine Learning DevOps

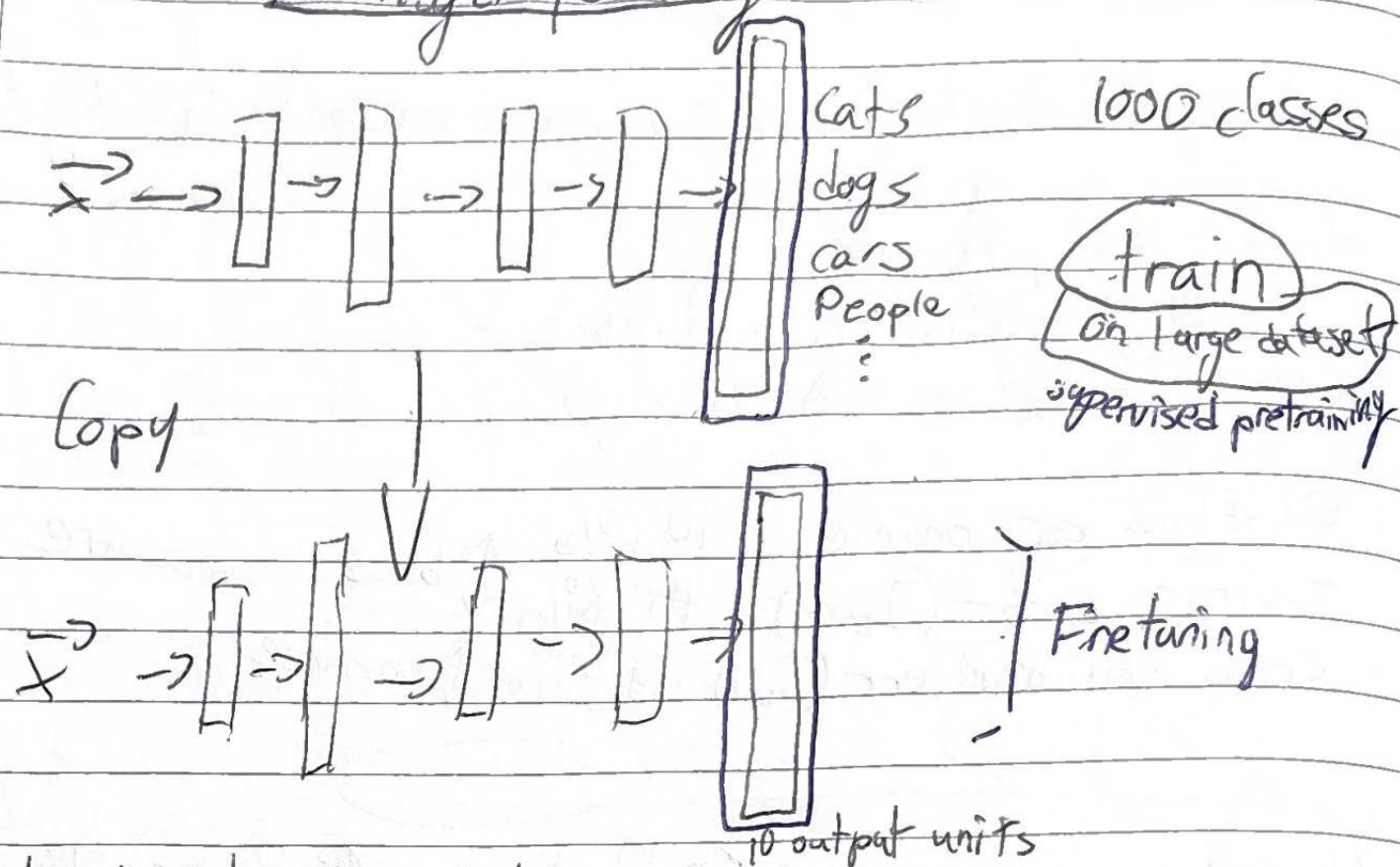


You can augment data to create more data like if its an img, rotating it with meaning etc.



Data engineering is very fruitful in modern ai.

Transfer learning



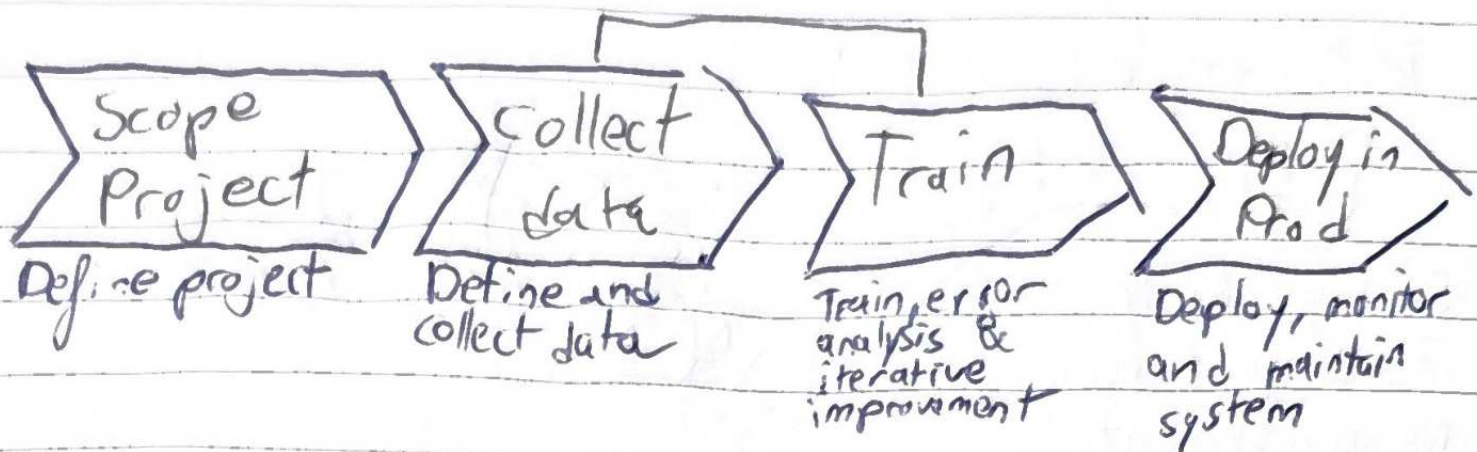
Option 1: Only train output layers parameters

Option 2: train all parameters

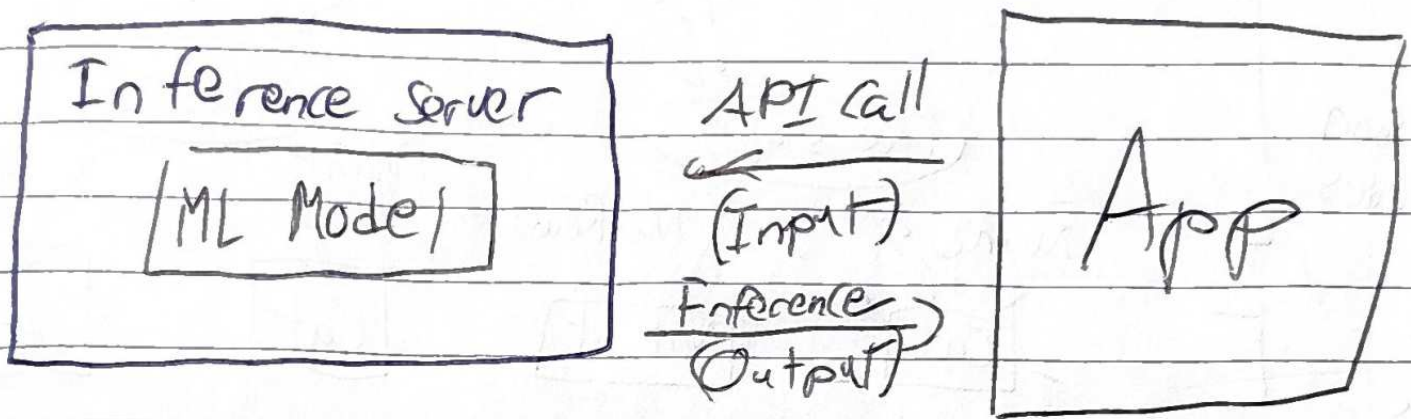
Summary

- ① Download neural network parameters pretrained on a large dataset with same input type (e.g., images, audio, text) as your application (or train your own).
- ② Further train (Fine tune) the network on your own data

Full cycle



Deployment



Software engineering may be needed for:

- reliable and efficient predictions
- scaling
- logging
- system monitoring
- Model updates

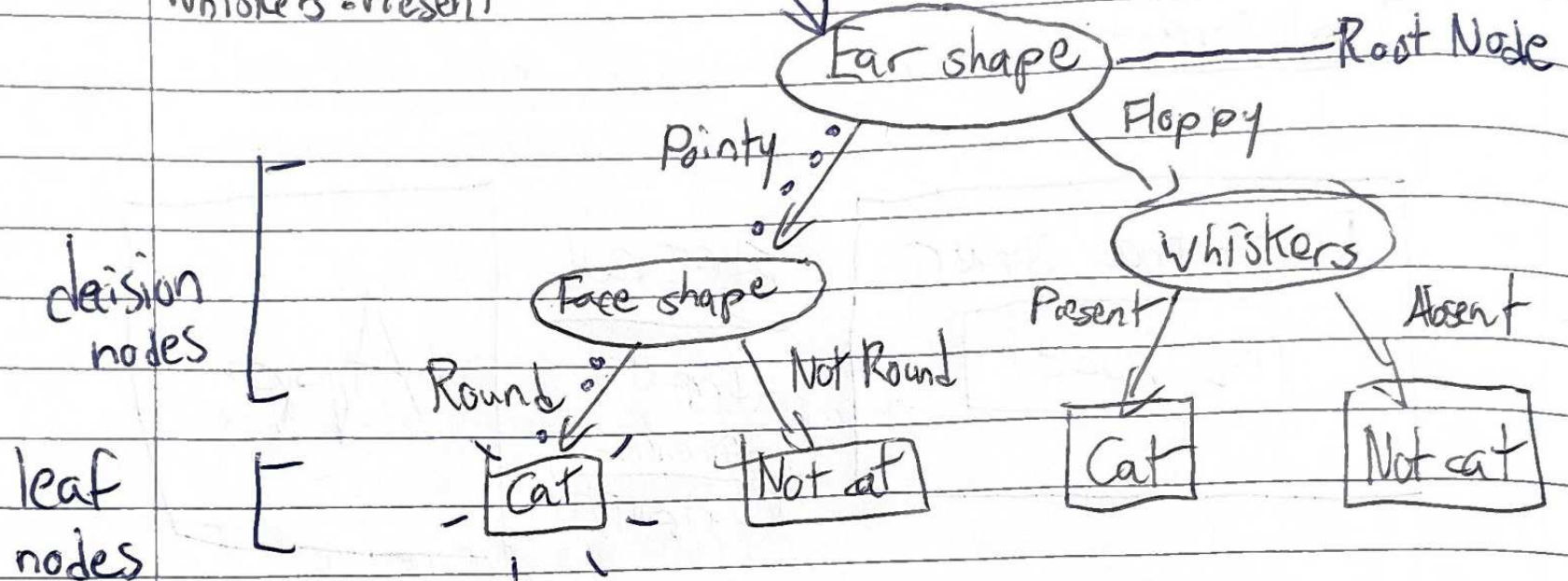
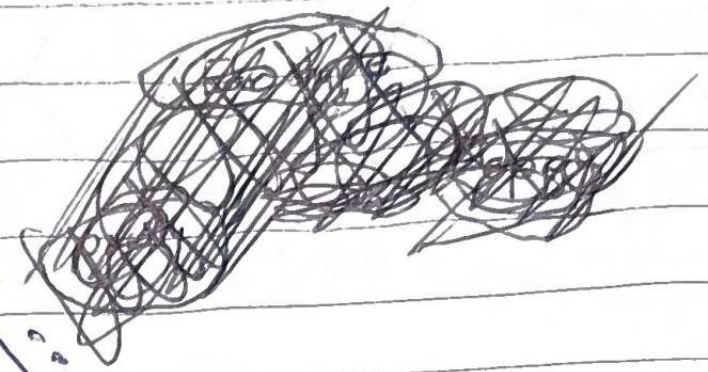
MLOps

Decision Tree

Test sample



Ear Shape: Pointy
Face Shape: Round
Whiskers: Present



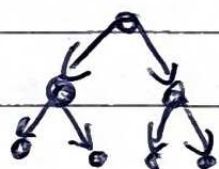
Decision 1: How to choose feature to split at each node?

Maximise purity (or minimise impurity)

Decision 2: When do you stop splitting?

you don't want to overfit

- When a node is 100% one class
- When splitting a node will result in tree exceeding a max depth
- When improvements in purity score are below a threshold
- When number of examples in a node is below a threshold



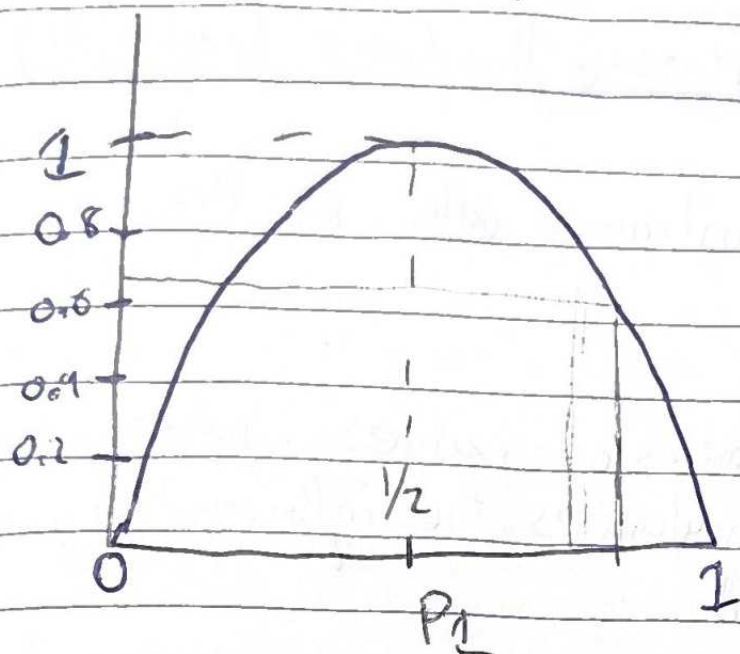
Recursive Algorithm

(code calls on itself)

Measuring Purity

Entropy as a measure of impurity

p_1 = fraction of examples that are cats



$H(p_1)$ 3 cats, 3 dogs ($3/6$)

$p_1 = 1/2$, $H(p_1) = 1$

5 cats, 1 dog ($5/6$)

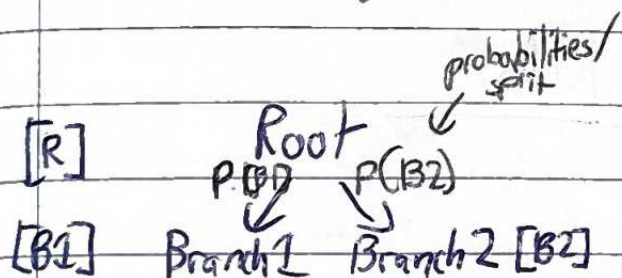
$p_1 = 5/6$, $H(p_1) = 0.65$

$$p_0 = 1 - p_1$$

$$\begin{aligned} H(p_1) &= -p_1 \log_2(p_1) - p_0 \log_2(p_0) \\ &= -p_1 \log_2(p_1) - (1-p_1) \log_2(1-p_1) \end{aligned}$$

Choosing a split

" $0 \log 0$ " is undefined
so just let it be 0



$$H(p_1^{[R]}) - \left(P(B1) H(p_1^{[B1]}) + P(B2) H(p_1^{[B2]}) \right)$$

= Information gain

Decision Tree learning

- Start with all examples at root node
- Calculate information gain for all possible features, and pick the one with the highest information gain.
- Split dataset according to selected feature, create left and right branches
- Keep repeating splitting process until stopping criteria met.

One hot encoding

If a categorical feature can take on ' K ' ~~values~~ values, create ' K ' binary features (0 or 1)

Decision tree $\rightarrow$ Continuous features (not 0/1)

The algorithm sorts the continuous values of the feature in ascending order

It identifies adjacent pairs of values where the target class changes and calculates the information gain for every candidate threshold

The algorithm calculates the Info gain for every candidate threshold.

Threshold with max info is chosen

Decision Trees VS NN

Decision Trees and Tree ensembles

- Works well on tabular (structured data) data
- Not recommended for unstructured data (images, audio, text)
- Fast
- Small decision trees may be human interpretable.

Neural Networks

- Works well on all types of data, including structured and not.
- May be slower
- Works with transfer learning
- * When building a system of multiple models working together, it might be easier to string together multiple NNs

Supervised learning

given both features x and y

Unsupervised learning

Training set $\rightarrow$ only x

Clustering $\rightarrow$ Applications

$K \neq K!$

small and big K

- Grouping similar news
- DNA analysis
- Astronomical data analysis
- Market segmentation

K-means algorithm

(1) randomly initialize K cluster centroids $\mu_1, \mu_2, \dots, \mu_K$

(2) Repeat {

assign pts to cluster centroids
for $i = 1$ to m

$c^{(i)} := \text{index (from 1 to } K) \text{ of cluster centroid closest to } x^{(i)}$
 $\min_k \|x^{(i)} - \mu_k\|^2$

move cluster centroids
for $k = 1$ to K

$\mu_k := \text{avg of pts assigned to cluster } k$
}

K-means optimisation objective

$c^{(i)} = \text{index of cluster } (1, 2, \dots, K) \text{ to which example } x^{(i)} \text{ is currently assigned}$

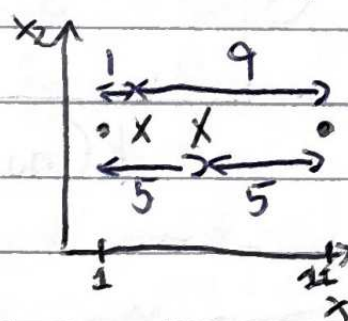
$\mu_k = \text{cluster centroid } k$

$\mu_c^{(i)} = \text{cluster centroid of cluster to which example } x^{(i)} \text{ has been assigned}$

∴ Cost function is ∴

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$
$$\min_{c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K} J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K)$$

Moving a centroid



$$\frac{1}{2}(1^2 + 9^2) = \frac{1}{2}(1 + 81) = 41$$

$$\frac{1}{2}(1 + 11) = 6$$

$$\frac{1}{2}(5^2 + 5^2) = 25$$

Initialising K-means

- ↳ Choose $K < m$
- ↳ Randomly pick K training examples
- ↳ Set $\mu_1, \dots, \mu_K$ equal to these K examples

Random initialisation

For $i = 1$ to 100 {

Randomly initialize K-means.

Run K-means. Get $c^{(1)}, \dots, c^{(m)}, \mu_1, \mu_2, \dots, \mu_K$

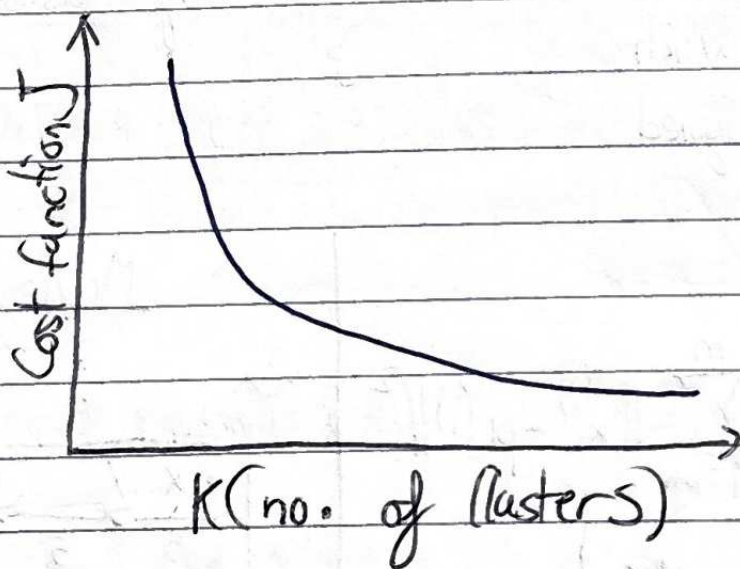
Compute cost function (distortion)

$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \mu_2, \dots, \mu_K)$

}

Pick set of clusters that gave lowest cost J

Choosing no. of clusters



the right K is often ambiguous.

Don't choose K just to minimize J !

Often, you want to get clusters for some later (downstream) purpose

Evaluate K-means based on how well it performs in that later purpose

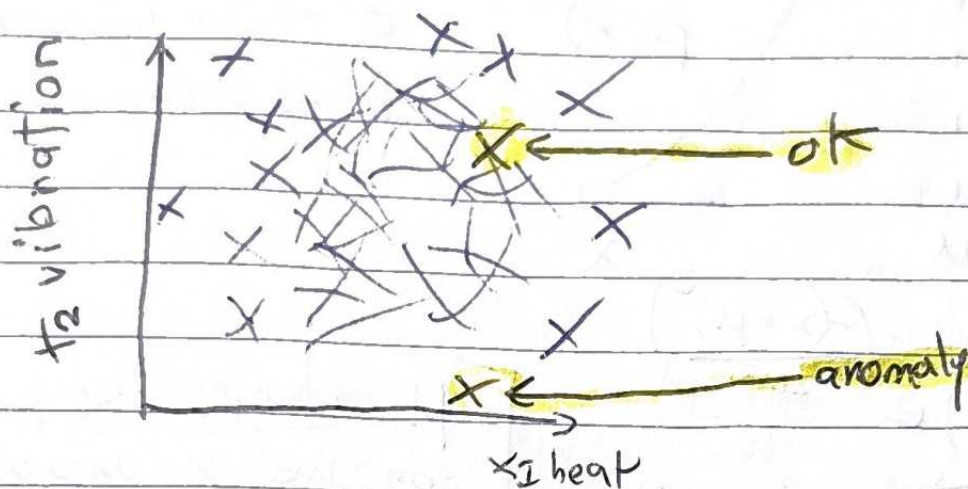
Anomaly Detection Example

Aircraft engine features:

x_1 = heat generated
 x_2 = vibration intensity
...

Dataset: $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$

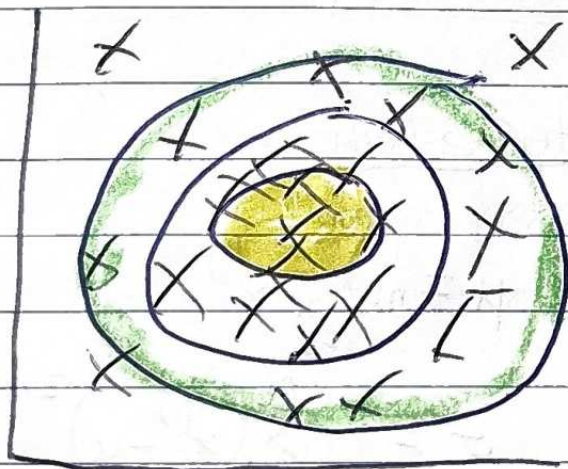
New engine: x_{test}



Density estimation

Dataset: $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$

Probability of x being seen in data set



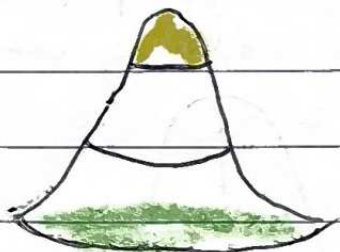
low probability

high probability

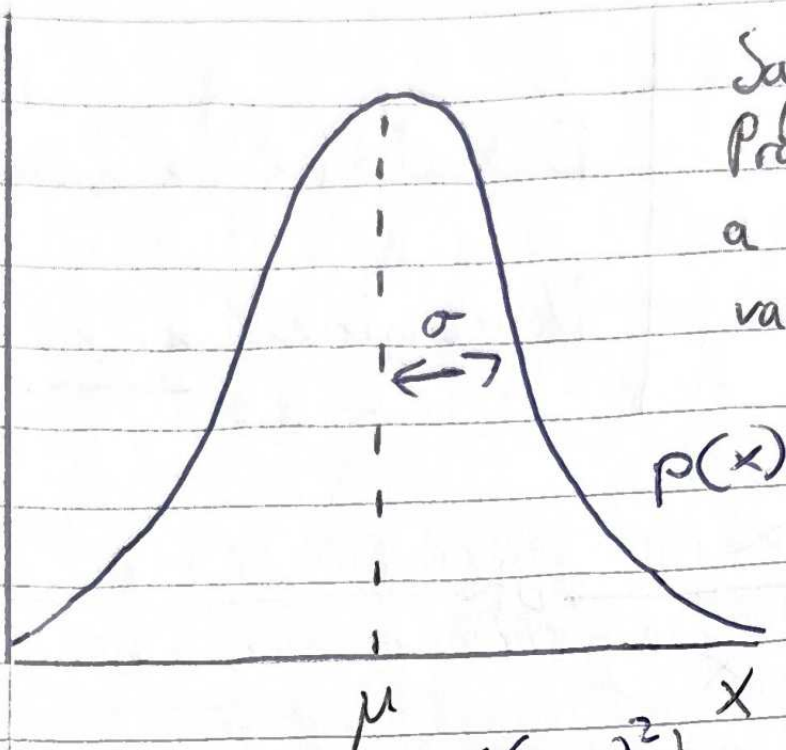
epsilon

$p(x_{\text{test}}) < \epsilon \rightarrow$ low

$p(x_{\text{test}}) \geq \epsilon \rightarrow$ high



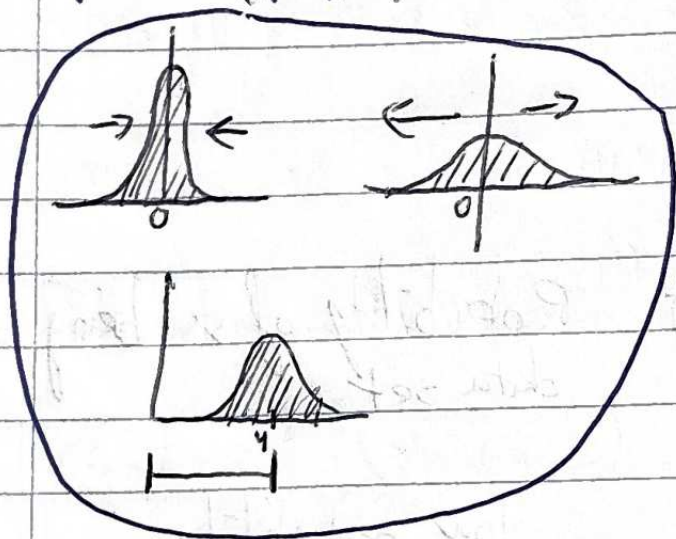
Gaussian (Normal distribution)



Say x is a n.r.
Probability of x is determined by
a gaussian with mean μ ,
variance σ^2

$\sigma \rightarrow$ standard deviation
 $\sigma^2 \rightarrow$ variance

$$p(x) = \left(\frac{1}{\sqrt{2\pi}\sigma} \right) e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$



μ changes the position
on the x axis of this
curve and σ changes the
squish of it. Its either really
tall or very wide etc.
Area is always 1!

Parameter estimation

Dataset: $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$

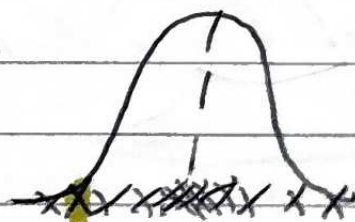


$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2$$

These formulae are called the maximum likelihood for μ, σ

* $\frac{1}{\sqrt{m-1}}$ may be preferred by statisticians.
doesn't really matter by a huge
factor

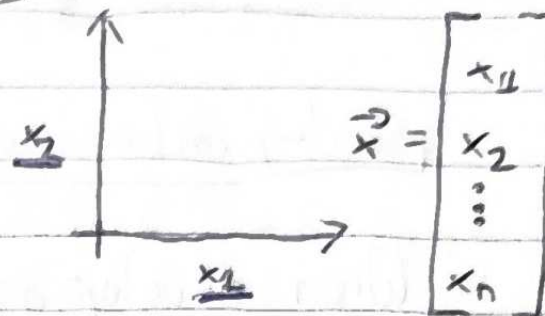


anomalous behaviour

Anomaly detection algorithm

Density estimation

Training set: $\{\vec{x}^{(1)}, \vec{x}^{(2)}, \dots, \vec{x}^{(m)}\}$



$$p(\vec{x}) = p(x_1; \mu_1, \sigma_1^2) * p(x_2; \mu_2, \sigma_2^2) * \dots * p(x_n; \mu_n, \sigma_n^2)$$
$$= \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) \quad \begin{matrix} \text{"add"} \\ \text{"multiply"} \end{matrix}$$

1. Choose n features x_i that you think might be indicative of anomalous examples

2. Fit parameters $\mu_1, \dots, \mu_n, \sigma_1^2, \dots, \sigma_n^2$

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)} \quad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2$$

vectorized formula $\left[\vec{\mu} = \frac{1}{m} \sum_{i=1}^m \vec{x}^{(i)} \quad \vec{\mu} = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix} \right]$

3. Given new example x , compute $p(x)$:

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) = \prod_{j=1}^n \left(\frac{1}{\sqrt{2\pi}\sigma_j} \right) e^{-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}}$$

Anomaly if $p(x) < \epsilon$



Developing and evaluating an anomaly detection system

Real-number evaluation

When developing a learning algorithm (choosing features etc.), making decisions is much easier if we have a way of evaluating our learning algorithm

Assume we have some labeled data, of anomalous and non-anomalous examples

$$y=1 \quad y=0$$

Training set: $x^{(1)}, x^{(2)}, \dots, x^{(m)}$ (assume normal/not anomalous examples)

$y=0$ for all training examples

Cross val set: $(x_{cv}^{(1)}, y_{cv}^{(1)}), \dots, (x_{cv}^{(m_{cv})}, y_{cv}^{(m_{cv})})$

Test set: $(x_{test}^{(1)}, y_{test}^{(1)}), \dots, (x_{test}^{(m_{test})}, y_{test}^{(m_{test})})$

include a few anomalous examples
 $y=1$

(mostly normal examples)
 $y=0$

On cross val / test example x , predict

$$y = \begin{cases} 1 & \text{if } p(x) < \epsilon \text{ (anomaly)} \\ 0 & \text{if } p(x) \geq \epsilon \text{ (normal)} \end{cases}$$

Possible evaluation metrics

- True Positive, False Positive, False negative, True negative matrix
- Precision recall
- F_1 - score

Anomaly Detection vs. Supervised Learning

Very small number of positive examples ($y=1$). (0-20 is common). Large number of negative ($y=0$) examples.

Large nr of positive and negative examples.

"20 positive examples"

Many different "types" of anomalies. Hard for any algorithm to learn from positive examples what anomalies may look like; future anomalies may look nothing like any of the anomalous examples we've seen so far

Enough positive examples for algorithm to get a sense of what positive examples are like, future positive examples likely to be similar ones to ones in training set.

Eg. Fraud

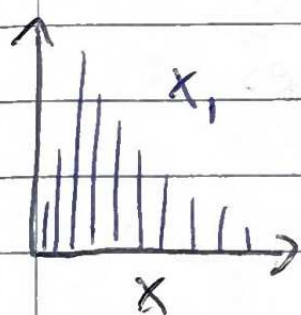
Eg. Spam

Anomaly $\rightarrow$ Features

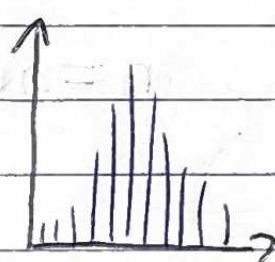
Make non-gaussian shaped features and transform them

plt.hist(x)

$$x_1 \leftarrow \log(x_i)$$



$\xrightarrow{\text{np.log}(x)}$



$\frac{\log(x+c)}{\log(x+c)}$
is a formula that can be used to make a feature more gaussian

other transforms $\rightarrow \sqrt{x} / \sqrt[3]{x}$

Apply same to CrossVal/train

Making recommendations

Predicting movie ratings

User rates movies using 0 → 5 stars

Ratings				
*				
*	*			
*	*	*		
*	*	*	*	
*	*	*	*	*

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)
Love at last	5	5	0	0
Romance forever	5	?	?	0
Cute puppies of bob	?	4	0	?
Nonstop car chases	0	0	5	4
Swords vs. Karate	0	0	5	?

n_u = no. of users

$r(i,j) = 1$ if user j has rated movie i

n_m = no. of movies

$y^{(i,j)}$ = rating given by user j to movie i (defined only if $r(i,j) = 1$)

(Dave)	x_1 (romance)	x_2 (action)
	0.9	0
	1.0	0.01
	0.99	0
	0.1	1.0
	0	0.9

What if we added features of movies?

n = no. of features

$$x^{(1)} = \begin{bmatrix} 0.9 \\ 0 \end{bmatrix}$$

$$x^{(3)} = \begin{bmatrix} 0.99 \\ 0 \end{bmatrix}$$

just linear regression

For user 1 (alice): Predict rating for movie i as $w \cdot x^{(i)} + b$

$$w^{(1)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix} \quad b^{(1)} = 0 \quad x^{(3)} = \begin{bmatrix} 0.99 \\ 0 \end{bmatrix} \rightarrow w^{(1)} \cdot x^{(3)} + b^{(1)} = 4.95$$

For user j : Predict user j 's rating for movie i : $w^{(j)} \cdot x^{(i)} + b^{(j)}$

Cost function

Notation:

- $r(i,j) = 1$ if user j has rated movie i (0 otherwise)
- $y^{(i,j)}$ = rating given by user j on movie i (if defined)
- $w^{(j)}, b^{(j)}$ = parameters for user j
- $x^{(i)}$ = feature vector for movie i

For user j and movie i , predict rating: $w^{(j)} \cdot x^{(i)} + b^{(j)}$

$m^{(j)}$ = no. of movies rated by user j
To learn $w^{(j)}, b^{(j)}$

$$\frac{1}{2m^j} \sum_{\substack{i: r(i,j)=1 \\ \text{if rated}}} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 = \min_{w^{(j)}, b^{(j)}} J(w^{(j)}, b^{(j)})$$

$$+ \frac{\lambda}{2m^j} \sum_{k=1}^n (w_k^{(j)})^2 \quad \text{note that } \frac{\lambda}{2m^j} \text{ cancels with } \frac{1}{2m^j}$$

Regularisation

To learn parameters $w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \dots, w^{(n_u)}, b^{(n_u)}$ for all users:

$$J(w^{(1)}, \dots, w^{(n_u)}, b^{(1)}, \dots, b^{(n_u)}) = \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i: r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

What if we don't have any x_1, x_2 ?

$$\begin{cases} w^{(1)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, w^{(2)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, w^{(3)} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}, w^{(4)} = \begin{bmatrix} 0 \\ 5 \end{bmatrix} \end{cases}$$

$$\begin{cases} b^{(1)} = 0, b^{(2)} = 0, b^{(3)} = 0, b^{(4)} = 0 \text{ using } w^{(j)} \cdot x^{(i)} \end{cases}$$

$$\left. \begin{aligned} w^{(1)} \cdot x^{(1)} &\approx 5 \\ w^{(2)} \cdot x^{(1)} &\approx 5 \\ w^{(3)} \cdot x^{(1)} &\approx 0 \\ w^{(4)} \cdot x^{(1)} &\approx 0 \end{aligned} \right\} \rightarrow x^{(1)} = \begin{bmatrix} 4 \\ 0 \end{bmatrix}$$

x_1 romance	x_2 action
?	?
?	?
?	?
?	?
?	?

Cost function

Given $w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \dots, w^{(n_u)}, b^{(n_u)}$
↳ to learn $x^{(i)}$:

$$J(x^{(i)}) = \frac{1}{2} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (x_k^{(i)})^2$$

↳ to learn $x^{(1)}, x^{(2)}, \dots, x^{(n_m)}$:

$$J(x^{(1)}, x^{(2)}, \dots, x^{(n_m)}) = \frac{1}{2} \sum_{i=1}^{n_m} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

Collaborative filtering

Putting everything together:

min
 $w^{(1)}, \dots, w^{(n_u)}$
 $b^{(1)}, \dots, b^{(n_u)}$
 $x^{(1)}, \dots, x^{(n_m)}$

$$J(w, b, x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

Gradient

Descent for 3 params!

$$w_j^{(t)} = w_j^{(t-1)} - \alpha \frac{d}{dw_j^{(t)}} \cdot (J(x, w, b))$$

$$b_j^{(t)} = b_j^{(t-1)} - \alpha \frac{d}{db_j^{(t)}} \cdot (J(x, w, b))$$

$$x_k^{(i)} = x_k^{(i-1)} - \alpha \frac{d}{dx_k^{(i)}} \cdot (J(x, w, b))$$

Binary labels

1 denotes a like

0 not engaged after being shown item

? hasn't interacted with item yet (not seen)

Movie	Alice (1)	Bob (2)	Carol (3)	Dave (4)
Love at last	1	1	0	0
Romance forever	1	?	?	0
Cute puppies of love	?	1	0	?
Nonstop car chases	0	0	1	1
Swords vs. Karate	0	0	1	?

Previously:

Predict $y^{(i,j)}$ as $w^{(j)} \cdot x^{(i)} + b^{(j)}$

linear regress

For binary labels:

Predict that the probability of $y^{(i,j)} = 1$ is given by $g(w^{(j)} \cdot x^{(i)} + b^{(j)})$

where $g(z) = 1 / (1 + e^{-z})$

logistic regress

Cost function for binary application - labels

$$J(w, b, x) = \sum_{(i,j): r(i,j)=1} L(\underbrace{f_{(w,b,x)}(x)}_{g(w^{(j)} \cdot x^{(i)} + b^{(j)})}, y^{(i,j)})$$

cost for all examples

Users who have not rated any movies

Eve

? This means that

? $w^{(5)} = [0]$

? $b^{(5)} = 0$

? $w^{(5)} \cdot x^{(i)} + b^{(5)} = 0$

? Not great

This assumes the user doesn't like any movies

5	5	0	0	?
5	2	2	0	?
2	4	0	?	?
0	0	5	4	?
0	0	5	0	?

Mean normalisation

→	5 5 0 0 ?	2.5	2.5	2.5 2.5 -2.5 -2.5
→	5 ? ? 0 ?	2.5	2.5	2.5 ? ? -2.5
	? 4 0 ? ?	2	2	? 2 -2 ?
	0 0 5 4 ?	2.25	2.25	-2.25 -2.25 2.75 1.75
	0 0 5 0 ?	1.25	1.25	-1.25 -1.25 3.75 -1.25

$\mu =$

$$y^{(i,j)} \downarrow w^{(i)}, b^{(i)}, x^{(i)}$$

For user i on movie j predict:

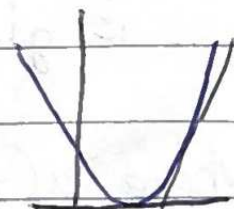
$$w^{(i)} \cdot x^{(j)} + b^{(i)} + \mu_i$$

User 5 (Eve)

$$w^{(5)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad b^{(5)} = 0 \quad \underbrace{w^{(5)} \cdot x^{(1)} + b^{(5)}}_0 + \mu_1 = 2.5$$

Tensorflow Implementation

Custom training loop for $J = (wx - 1)^2$



Fix $b=0$ for this example

$w = \text{tf.Variable}(3.0)$ variable to optimize

$x = 1.0$

$y = 1.0$

iterations = 30

for iter in range(iterations): record steps
with `tf.GradientTape()` as tape:

$$fwb = w * x$$

$$\text{costJ} = (fwb - y)^2$$

$[dJdw] = \text{tape.gradient}(\text{costJ}, [w])$ Calculate gradients

$w.\text{assign_add}(-\text{alpha} * dJdw)$ Run one step of gd by updating

$$\frac{\partial}{\partial w} J(w)$$

Collaborative filtering vs Content-based filtering

→ Collaborative filtering:

Recommend items to you based on ratings of users who gave similar ratings as you

→ Content-based filtering:

Recommend items to you based on features of user and item to find good match

$r(i, j) = 1$ if user j has rated item i

$y_{(i, j)}$ rating given by user j on item i (if defined)

Examples of user & item features

User features:

- Age
- Gender
- Country
- Movies watched
- Avg rating per genre
- ...

feature vector

$x_u^{(j)}$ for user j

Vector size
could be
different

Movie features:

- Year
- Genre/genres
- Reviews
- Average rating
- ...

feature vector

$x_m^{(i)}$ for movie i

Content-based filtering: learning to match

Predict rating of user j on movie i as

$$w^{(j)} \cdot x^{(i)} + b^{(j)}$$

$$v_u^{(j)} \cdot v_m^{(i)}$$

computed from $x_u^{(j)}$

computed from $x_m^{(i)}$

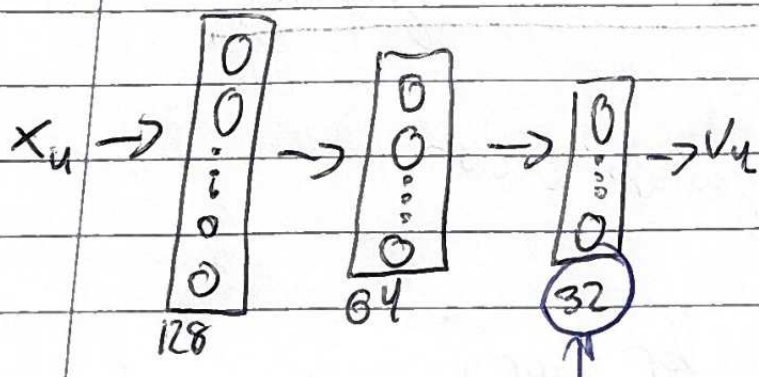
user's preferences $v_u = \begin{bmatrix} 4.9 \\ 0.1 \\ \vdots \\ 3.0 \end{bmatrix}$ likes

$$\frac{x_u^{(j)}}{x_m^{(i)}}$$

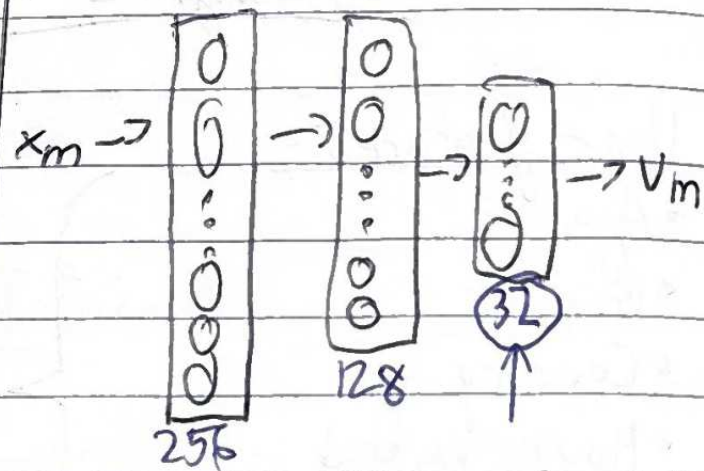
movie features $v_m = \begin{bmatrix} 4.5 \\ 0.2 \\ \vdots \\ 3.5 \end{bmatrix}$ romance action

Neural Network architecture

$x_u \rightarrow v_u$ User Network

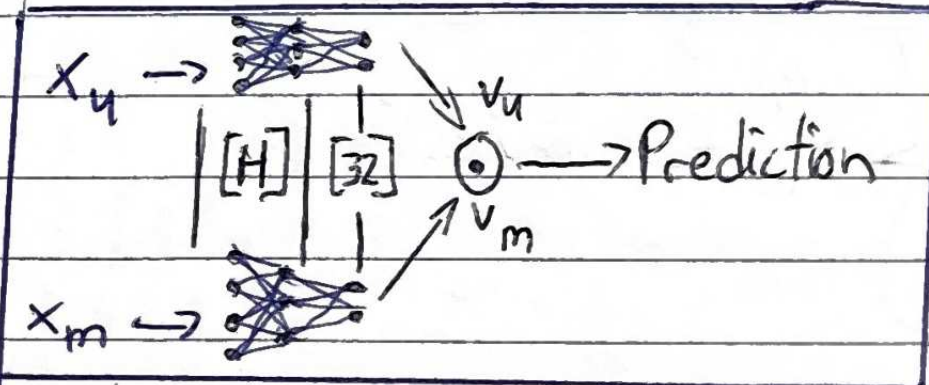


$x_m \rightarrow v_m$ Movie Network



$$\text{Prediction} = v_u^{(j)} \cdot v_m^{(i)}$$

$g(v_u^{(j)} \cdot v_m^{(i)})$ to predict the probability that $y^{(i,j)}$ is 1



Cost Function $J = \sum_{(i,j): r(i,j)=1} (v_u^{(j)} \cdot v_m^{(i)} - y^{(i,j)})^2 + \text{NN regularisation}$

Learned user and item vectors

→ $v_u^{(i)}$ is a vector of length 32 that describes user j with features $x_u^{(j)}$

→ $v_m^{(i)}$ is a vector of length 32 that describes movie i with features $x_m^{(i)}$

To find movies similar to movie i : $\|v_m^{(k)} - v_m^{(i)}\|^2$ small

Distance formula

$$\|x^{(k)} - x^{(i)}\|^2$$

Note: This can be ^(pre)computed ahead of time

↳ Prod

Many feature vectors can be large like movies on Netflix, songs on Spotify etc. Precomputing is essential



Two Steps: Retrieval & Ranking

Retrieval:

- Generate large list of plausible item candidates

eg.

1) For each of the last 10 movies watched by user, find 10 most similar movies

2) For most viewed 3 genres, find the top 10 movies

3) Top 20 movies in the country

- Combine retrieved items into list, removing duplicates and items already watched/purchased

Ranking:

- Take list retrieved and rank using learned model

- Display ranked items to user



Retrieval step

- Retrieving more items results in better performance, but slower recommendations.
- To analyze/optimize the tradeoff, carry out offline experiments to see if retrieving additional items results in more relevant recommendations (ie. $p(y^{(i,j)})=1$ of items displayed to user are higher).

Tensorflow implementation

Mathematics! → Vectors

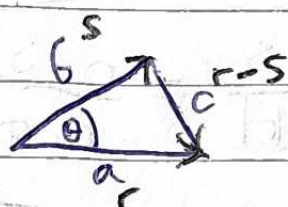
Dot product:

$$\left. \begin{aligned} r \cdot s &= r_i s_i + r_j s_j \\ r \cdot s &= s \cdot r \end{aligned} \right\} \text{Commutative}$$

$$r \cdot (s + t) = r \cdot s + r \cdot t \quad] \text{ distributive}$$

$$\left. \begin{aligned} r \cdot (as) &= a(r \cdot s) \\ r_1(as_1) + r_2(as_2) &= a(r_1s_1 + r_2s_2) \\ &= a(r \cdot s) \end{aligned} \right\} \text{associative}$$

$$c^2 = a^2 + b^2 - 2ab \cos \theta$$

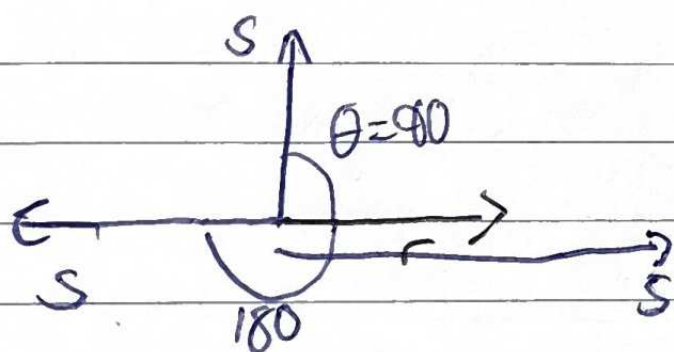


cosine rule for vectors

$$\begin{aligned} r \cdot r &= r_1 r_1 + r_2 r_2 \\ &= r_1^2 + r_2^2 \\ &= (\sqrt{r_1^2 + r_2^2})^2 \\ &= |r|^2 \end{aligned}$$

$$|r - s|^2 = |r|^2 + |s|^2 - 2|r||s|\cos \theta$$

$$\begin{aligned} (r - s) \cdot (r - s) &= r \cdot r - s \cdot r - s \cdot r - s \cdot s \\ &= |r|^2 - 2rsr + |s|^2 \end{aligned}$$



$$r \cdot s = |r||s|\cos \theta$$

$$\cos 90^\circ = 0$$

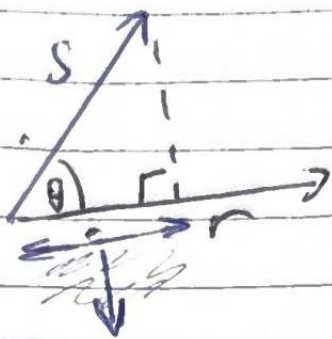
$$r \cdot s = |r||s| \cdot 0$$

$$\cos 0^\circ = 1 \quad r \cdot s = |r||s|$$

$$\cos 180^\circ = -1 \quad s \cdot s = -|r||s|$$

When dot product is 0 they're at 90° , same way they are positive opposite directions negative

Projection



$$\cos \theta = \frac{\text{adj}}{\text{hyp}} = \frac{\text{adj}}{|s|}$$

$$r \cdot s = |r| \underbrace{|s| \cos \theta}_{\text{adj}}$$

$$\therefore \frac{r \cdot s}{|r|} = |s| \cos \theta$$

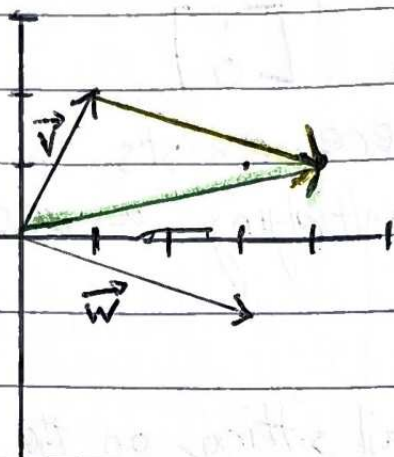
Scalar projection

$$r \cdot s = |r| \times \text{projection}$$

vector
projection

$$r \frac{r \cdot s}{|r|} = \frac{r \cdot s}{r \cdot r} r$$

Vector addition



$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} x_1 + x_2 \\ y_1 + y_2 \end{bmatrix}$$

move origin of either vector to the other's tip. The new tip is the tip of their sum ($\vec{v} + \vec{w}$)

"Linear combination"
of $\vec{v}$ and $\vec{w}$

almost the same extension as adding
on a nr line in terms of steps

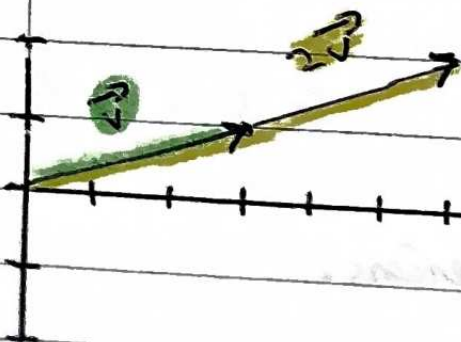
$$a\vec{v} + b\vec{w}$$

Multiplying a vector by a nr

When multiplying by a nr you are stretching/squishing its length
Negative numbers flip the vector, then apply ratio.
"Scaling"

These nrs acting on a vector are "scalars"

$$2 \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$$

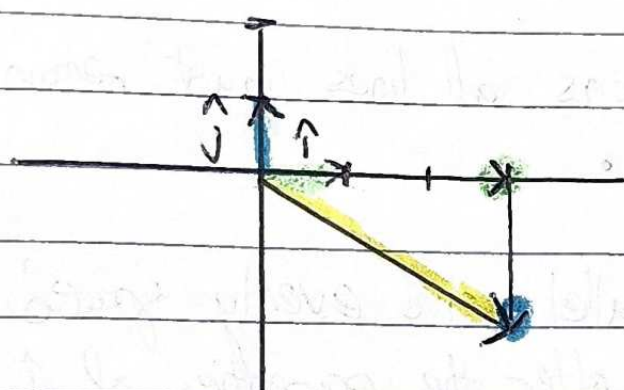


$\hat{i}$ and $\hat{j}$

it's better to think of each coordinate of a vector as a scalar in its own right. Thinking about how each one stretches and squishes vectors

in the x, y plane there exists $\hat{i}$ and $\hat{j}$ each of length 1. $\hat{j}$ being vertical and $\hat{i}$ horizontal. these are the unit vectors.

In the case $\begin{bmatrix} 3 \\ -2 \end{bmatrix}$ is the same as $(3)\hat{i} + (-2)\hat{j}$

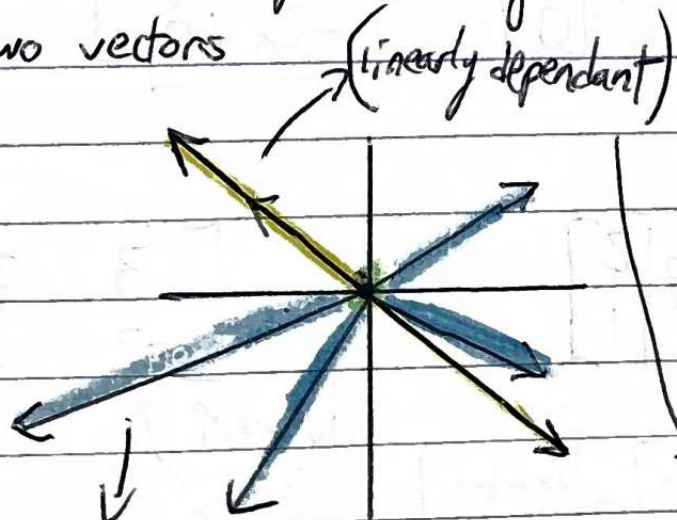


these are the basis of the coord system

Every time we describe vectors numerically it depends implicitly on choice of what basis vectors we use.

The set of all possible vectors you can reach with a linear combination of a given pair of vectors is called the "span" of those two vectors

Span usually reaches all 2D space unless vectors line up or are equal to 0



$$a\vec{v} + b\vec{w}$$

let a and b vary over all real nos

$$\vec{u} = a\vec{v} + b\vec{w}$$

(one is a scaled ver of the other)

linearly independant adding to span
 $\vec{v} + \vec{v} + \vec{v}$

Linear transformation

function
Input $\rightarrow f(x) \rightarrow$ Output

Why not just call it a function? The word Transformation suggests you think using movement.

$$\begin{bmatrix} 5 \\ 7 \end{bmatrix} \xrightarrow{L(\vec{v})} \begin{bmatrix} 2 \\ -3 \end{bmatrix}$$

Vector Input

Vector Output

Every point in space moves to some other point. Squishing and morphing space itself

In linear transformations all lines must remain lines and the origin doesn't move.

Grids remain parallel and evenly spaced $\therefore$ linear transformations just alter the properties of $\hat{i}$ and $\hat{j}$ for all vectors $\vec{v} = a\hat{i} + b\hat{j}$

if the vectors $\hat{i}$ and $\hat{j}$ land on are linearly dependant all vectors from 2D space are squished onto a line

for an $\hat{i} \rightarrow \begin{bmatrix} 1 \\ -2 \end{bmatrix}$

$\hat{j} \rightarrow \begin{bmatrix} 3 \\ 0 \end{bmatrix}$

2x2 matrix

can be made for $\hat{i}$ $\hat{j}$

$$\begin{bmatrix} x \\ y \end{bmatrix} \rightarrow x \begin{bmatrix} 1 \\ -2 \end{bmatrix} + y \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 1x + 3y \\ -2x + 0y \end{bmatrix}$$

$$\begin{bmatrix} 5 \\ 7 \end{bmatrix} = 5 \begin{bmatrix} 3 \\ -2 \end{bmatrix} + 7 \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 3 \\ -2 \end{bmatrix} \quad \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

where $\hat{i}$ lands

where $\hat{j}$ lands

$$\begin{bmatrix} x \\ y \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$